Essential C++

A clear guide to the language developers use every day

From C to modern C++: `const` and `constexpr`, references, classes, inheritance, polymorphism, templates, move semantics, smart pointers and the STL

Prepared for Hrishikesh Tiwari

Built around the IIT Kharagpur CS20006 Software Engineering syllabus (Spring 2024) and the topic structure of learncpp.com

November 2026

Contents

1 Before You Begin **7**

1.1 How to use this guide . . . 7

1.2 Compiling a program . . . 7

1.3 What happens when you build: preprocessor, compiler, linker . . . 8

1.3.1 Declarations, definitions and header files . . . 8

1.4 Undefined behaviour . . . 9

2 From C to C++ **10**

2.1 Hello, C++ . . . 10

2.2 What C++ adds to C at a glance . . . 11

2.3 Fundamental types . . . 11

2.4 Initialisation: the three forms . . . 11

2.5 `auto`: let the compiler deduce the type . . . 12

2.6 Type aliases . . . 12

2.7 Enumerations: prefer `enum class` . . . 12

2.8 Structs . . . 13

2.9 Strings . . . 13

2.10 Range-based for loops . . . 13

3 Constants: `const`, `constexpr` and `inline` **14**

3.1 `const` variables . . . 14

3.1.1 Why not `#define`? . . . 14

3.2 Compile time versus runtime . . . 14

3.3 `constexpr` variables . . . 15

3.4 `constexpr` functions . . . 15

3.4.1 `consteval` and `constinit` (C++20) . . . 16

3.5 `static_assert`: checks while compiling . . . 16

3.6 `if constexpr` . . . 16

3.7 Inline functions . . . 17

3.7.1 The original meaning: avoid function-call overhead . . . 17

3.7.2 The modern meaning: "may be defined in more than one file" . . . 17

3.7.3 Inline variables (C++17) . . . 17

4 References and Pointers **19**

4.1 Every object lives at an address . . . 19
4.2 Pointers . . . 19
4.2.1 Pointers and arrays . . . 20
4.3 References . . . 20
4.4 Passing arguments to functions . . . 20
4.4.1 Pass by const reference: the everyday default . . . 21
4.5 Returning references, and dangling . . . 21
4.6 `const` with pointers: read the declaration right to left . . . 21
4.7 lvalues and rvalues (a first look) . . . 22

5 Functions: Default Arguments, Overloading and More **23**
5.1 Default arguments . . . 23
5.2 Function overloading . . . 23
5.2.1 How the compiler chooses (overload resolution) . . . 23
5.3 Deleting functions . . . 24
5.4 Returning results . . . 24
5.4.1 Multiple values . . . 24
5.4.2 Maybe a value: `std::optional` . . . 24
5.4.3 Do not ignore the result: `[[nodiscard]]` . . . 25
5.5 Function pointers . . . 25
5.6 Lambdas in one page . . . 25
5.7 The call stack, and why recursion has limits . . . 25

6 Dynamic Memory Management **26**
6.1 Where objects live . . . 26
6.2 `new` and `delete` . . . 26
6.2.1 `new` versus `malloc` . . . 26
6.3 The classic bugs . . . 26

7 Classes and Objects **28**
7.1 Why object-oriented programming? . . . 28
7.2 A first class . . . 28
7.3 Access specifiers . . . 29
7.3.1 `class` versus `struct` . . . 29
7.4 Member functions defined outside the class . . . 29
7.5 The hidden `this` pointer . . . 29
7.6 Const objects and const member functions . . . 30
7.6.1 `mutable` . . . 30

7.7 Getters and setters, used wisely . . . 31

8 Constructors, Destructors and Copying **32**

8.1 Constructors . . . 32

8.1.1 The member initialiser list . . . 32

8.1.2 Default member initialisers and the default constructor . . . 32

8.1.3 Delegating constructors . . . 33

8.1.4 `explicit`: stop surprise conversions . . . 33

8.2 Destructors . . . 33

8.3 Object lifetime . . . 34

8.4 Copying objects . . . 34

8.4.1 Shallow versus deep copy: the classic disaster . . . 34

8.4.2 The Rule of Three, Five and Zero . . . 35

8.4.3 Disabling copying . . . 36

8.4.4 Copy elision . . . 36

8.5 Const-ness: the whole picture . . . 36

9 Static Members and Friends **38**

9.1 The many meanings of `static` . . . 38

9.1.1 Static local variables . . . 38

9.2 Static data members . . . 38

9.3 Static member functions . . . 39

9.4 Friend functions and friend classes . . . 39

10 Operator Overloading **41**

10.1 The rules . . . 41

10.2 Member or non-member? . . . 41

10.3 A complete example . . . 41

10.4 Input: `operator>>` . . . 42

10.5 Comparison operators and the spaceship (C++20) . . . 43

10.6 Increment and decrement . . . 43

10.7 Subscript and function-call operators . . . 43

10.8 Conversion operators . . . 44

10.9 Assignment . . . 44

11 Namespaces and Program Organisation **45**

11.1 The problem namespaces solve . . . 45

11.2 `using` declarations and `using` directives . . . 45

11.3 Argument-dependent lookup . . . 46
11.4 Linkage: what is visible across files . . . 46
11.5 Global variables . . . 46
11.6 A typical project layout . . . 46

12 Inheritance **47**
12.1 The idea: "is-a" . . . 47
12.2 Construction and destruction order . . . 48
12.3 `protected` members . . . 48
12.4 Public, protected and private inheritance . . . 48
12.5 Overriding and hiding . . . 48
12.6 Multiple inheritance . . . 49
12.6.1 The diamond problem and virtual base classes . . . 49
12.7 Composition versus inheritance . . . 50

13 Polymorphism and Virtual Functions **51**
13.1 Base-class pointers and references . . . 51
13.2 Virtual functions: dynamic binding . . . 51
13.2.1 `override` and `final` . . . 52
13.3 Virtual destructors . . . 52
13.4 Pure virtual functions and abstract classes . . . 53
13.5 Object slicing . . . 53
13.6 How it works: the virtual table . . . 53
13.6.1 The vtable and the vptr . . . 53
13.6.2 What this explains . . . 54
13.7 Runtime versus compile-time polymorphism . . . 54

14 Type Conversion and Casting **55**
14.1 Implicit conversions . . . 55
14.2 Why C-style casts are dangerous . . . 55
14.3 `static_cast` . . . 55
14.4 `dynamic_cast` . . . 56
14.5 `const_cast` . . . 56
14.6 `reinterpret_cast` . . . 57

15 Exceptions **58**
15.1 Why exceptions? . . . 58
15.2 `throw`, `try` and `catch` . . . 58

15.3 Stack unwinding: what happens on the way up . . . 58
15.4 The standard exception hierarchy . . . 59
15.5 Your own exception types . . . 60
15.6 Rethrowing . . . 60
15.7 `noexcept` . . . 61
15.8 Exception-safety guarantees . . . 61
15.9 When not to use exceptions . . . 61

16 Templates **62**
16.1 The problem templates solve . . . 62
16.2 Function templates . . . 62
16.2.1 Several type parameters . . . 62
16.2.2 Abbreviated function templates (C++20) . . . 63
16.3 Non-type template parameters . . . 63
16.4 Class templates . . . 63
16.4.1 Class template argument deduction (CTAD) . . . 64
16.5 Why templates live in header files . . . 64
16.6 Template specialisation . . . 64
16.7 Alias templates . . . 65
16.8 Variadic templates and fold expressions . . . 65
16.9 Concepts: constraining templates (C++20) . . . 65

17 Functors, Lambdas and `std::function` **67**
17.1 Function objects (functors) . . . 67
17.2 Lambdas . . . 67
17.2.1 Anatomy . . . 68
17.2.2 Captures . . . 68
17.2.3 Generic lambdas . . . 68
17.3 `std::function`: storing any callable . . . 69

18 Move Semantics and Smart Pointers **70**
18.1 The problem: needless copies . . . 70
18.2 Rvalue references . . . 70
18.3 Move constructor and move assignment . . . 70
18.3.1 The moved-from state . . . 71
18.4 `std::move` does not move anything . . . 72
18.4.1 When moves happen automatically . . . 72
18.4.2 Sink parameters: take by value, then move . . . 72

18.5 The Rule of Five, and why `noexcept` matters . . . 72
18.6 Forwarding references and `std::forward` . . . 72
18.7 Smart pointers . . . 73
18.7.1 `std::unique_ptr`: sole ownership . . . 73
18.7.2 `std::shared_ptr`: shared ownership . . . 73
18.7.3 `std::weak_ptr`: breaking cycles . . . 74
18.7.4 Choosing . . . 74

19 The Standard Library: Containers, Iterators, Algorithms and I/O **75**
19.1 Containers . . . 75
19.1.1 `std::vector` in depth . . . 75
19.1.2 Maps . . . 76
19.2 Iterators . . . 76
19.3 Algorithms . . . 76
19.3.1 Ranges (C++20) . . . 77
19.4 Other useful utilities . . . 77
19.5 Input and output streams . . . 78
19.5.1 Console and string streams . . . 78
19.5.2 Files . . . 78
19.6 Quick reference: big-O of common operations . . . 78

A From Design to Working Code **80**
A.1 Development life cycle models in brief . . . 80
A.2 UML class diagrams and their C++ meaning . . . 80
A.3 Testing your code . . . 80
A.3.1 Assertions . . . 80
A.3.2 Unit tests . . . 81

B Everyday Best-Practice Checklist **82**

C Where to Go Next **83**
C.1 Practice projects . . . 83
C.2 References . . . 83

Chapter 1

Before You Begin

1.1 How to use this guide

This guide explains the parts of C++ that working developers use every day, in plain language. It follows the order of IIT Kharagpur's CS20006 course (from C to C++, then classes, inheritance, polymorphism, casting, exceptions and templates) and fills in the modern features that course time does not reach: `constexpr`, move semantics, smart pointers, lambdas and the standard library. The overall map of topics follows the free tutorial site learncpp.com, which is the best place to go deeper on any section.

Three kinds of boxes appear throughout:

Key idea

The central idea of a section, in one or two sentences. If you remember nothing else, remember these.

Pitfall

A common mistake, usually one that compiles fine and fails at runtime.

Best practice

What experienced developers actually do.

The best way to learn from this guide: type every example yourself (do not copy-paste), compile it, then change it and predict what will happen before you run it.

1.2 Compiling a program

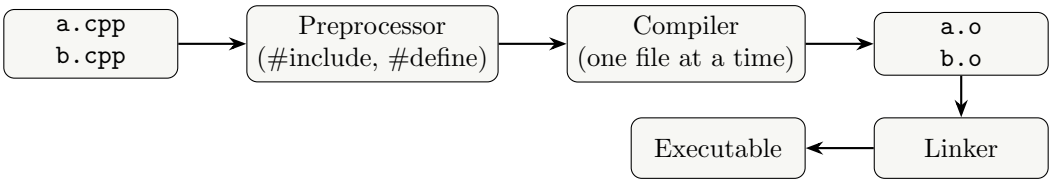
All examples use the C++20 standard. With the GCC or Clang compiler, a good everyday command is:

```
g++ -std=c++20 -Wall -Wextra -Wpedantic -g main.cpp -o main
./main
```

- `-std=c++20` chooses the language version.
- `-Wall -Wextra -Wpedantic` turn on most warnings. Treat warnings as bugs you have not met yet.
- `-g` adds debugging information so a debugger such as `gdb` can show your source code.
- While learning, add `-fsanitize=address,undefined`. These *sanitizers* catch memory errors and undefined behaviour at runtime and tell you the exact line. They are one of the most useful tools in C++.

1.3 What happens when you build: preprocessor, compiler, linker

Understanding the build pipeline explains many confusing error messages, so it is worth a few minutes.



1. **Preprocessor.** Runs first and works on text only. `#include <iostream>` literally pastes the contents of that header file into yours. `#define` does find-and-replace.
2. **Compiler.** Translates each `.cpp` file *separately* into an object file (`.o`). Each file plus everything it includes is called a *translation unit*. The compiler only needs to know that a function *exists* (its declaration) to compile a call to it.
3. **Linker.** Joins the object files and libraries, and connects every call to the one place where that function is actually defined.

This explains two classic errors. "*was not declared in this scope*" is a **compiler** error: you used a name before declaring it. "*undefined reference to ...*" is a **linker** error: you declared a function but never defined it, or forgot to compile the file that defines it.

1.3.1 Declarations, definitions and header files

A *declaration* tells the compiler a name and its type. A *definition* provides the actual body or storage.

```
int add(int a, int b);            // declaration (a "prototype")

int add(int a, int b) {           // definition
    return a + b;
}
```

Header files (`.h` or `.hpp`) hold declarations that several `.cpp` files need. Source files (`.cpp`) hold the definitions.

```
// math_utils.h
#pragma once                      // include this file only once per translation unit
int add(int a, int b);

// math_utils.cpp
#include "math_utils.h"
int add(int a, int b) { return a + b; }

// main.cpp
#include <iostream>
#include "math_utils.h"
int main() { std::cout << add(2, 3) << '\n'; }
```

Build both source files together: `g++ -std=c++20 main.cpp math_utils.cpp -o app`.

The One Definition Rule (ODR)

Something may be *declared* many times, but a function or global variable must be *defined* exactly once across the whole program. Defining a normal function in a header that two `.cpp` files include breaks this rule and gives a "multiple definition" linker error. (Templates, `inline` functions and class definitions are allowed in headers; you will see why later.)

`#pragma once` is a *header guard*: it stops the same header being pasted twice into one translation unit. The traditional, fully standard version does the same job with macros:

```
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
int add(int a, int b);
#endif
```

1.4 Undefined behaviour

C++ trusts you. If your program breaks certain rules (reading an uninitialised variable, going past the end of an array, using memory after freeing it, signed integer overflow), the standard says the behaviour is *undefined*. That does not mean "it crashes". It means anything can happen: it may seem to work today, print garbage tomorrow, and fail only in the optimised release build. Much of good C++ style exists to make undefined behaviour hard to write. Keep warnings on and use the sanitizers.

Chapter 2

From C to C++

C++ began as "C with classes", and almost every C program is also a valid C++ program. But C++ adds safer and more expressive ways of doing the same things. This chapter covers what changes first.

2.1 Hello, C++

```cpp
#include <iostream>   // input/output streams (replaces <stdio.h>)
#include <string>     // std::string

int main() {
    std::string name;
    std::cout << "What is your name? ";
    std::cin >> name;                          // reads one word
    std::cout << "Hello, " << name << "!\n";
    return 0;                                  // optional in main
}
```

- `std::cout` is the standard output stream; `<<` sends values into it. Unlike `printf`, there are no format specifiers to get wrong: the compiler knows each value's type.
- `std::cin` is the standard input stream; `>>` extracts a value from it.
- `std::` is a *namespace* prefix. Everything in the standard library lives in namespace `std` so that its names do not collide with yours (Chapter 11).
- Prefer `'\n'` over `std::endl`. `std::endl` also *flushes* the stream, which is slow when done in a loop.
- To read a whole line including spaces, use `std::getline(std::cin, name);`.

2.2 What C++ adds to C at a glance

C	C++ equivalent and why it is better
`printf`, `scanf`	`std::cout`, `std::cin`: type-safe and extensible to your own types
`char*` strings	`std::string`: manages its own memory, grows as needed
`malloc`/`free`	`new`/`delete`, and better still smart pointers and containers
Arrays + length variable	`std::vector` (growable) and `std::array` (fixed size)
`#define PI 3.14`	`constexpr double pi = 3.14;`: typed, scoped, debuggable
Pointers for "output" parameters	References (`int&`)
`NULL` or `0`	`nullptr`: a real null-pointer value with its own type
`int` flags 0/1	`bool` with `true` and `false`
Function name prefixes (`vec_add`)	Function overloading and namespaces
`struct` + free functions	Classes with member functions, access control, constructors
`void*` generic code	Templates: generic and type-safe
Error codes	Exceptions (plus error codes where they fit better)

2.3 Fundamental types

Type	Typical size	Use
`bool`	1 byte	`true`/`false`
`char`	1 byte	a single character such as `'a'`
`int`	4 bytes	default whole number
`long long`	8 bytes	large whole numbers
`float`	4 bytes	about 7 significant decimal digits
`double`	8 bytes	about 15–16 digits; the default for real numbers
`std::size_t`	8 bytes (64-bit)	sizes and indices; unsigned

The standard only guarantees *minimum* sizes, so an `int` is not promised to be 4 bytes everywhere. When the exact width matters (file formats, networking, hashing), use the fixed-width types from `<cstdint>`: `std::int32_t`, `std::int64_t`, `std::uint8_t`, and so on. Use `sizeof(x)` to ask for the size in bytes.

Unsigned integers wrap around

Unsigned types never go negative; they wrap. If `std::size_t n = 0;` then `n - 1` is a huge number (18446744073709551615 on 64-bit systems). A loop like `for (std::size_t i = v.size() - 1; i >= 0; --i)` never ends, because `i >= 0` is always true. Use signed integers for arithmetic, and be careful when mixing the two.

Floating point is approximate

`0.1 + 0.2 == 0.3` is `false`, because these numbers have no exact binary representation. Compare floating-point numbers with a tolerance: `std::abs(a - b) < 1e-9`.

2.4 Initialisation: the three forms

C++ has several ways to give a variable its first value:

```
int a = 5;      // copy initialisation (C style)
int b(5);       // direct initialisation
int c{5};       // list (brace) initialisation  <-- preferred
int d{};        // value initialisation: d is 0
int e;          // NOT initialised: reading e is undefined behaviour
```

Brace initialisation is preferred because it refuses *narrowing conversions*, where information would be lost silently:

```
int x = 3.9;    // compiles; x is 3 (fraction silently dropped)
int y{3.9};     // compile error: narrowing from double to int
```

Best practice

Always initialise variables, preferably with braces: `int count{};` or `double rate{0.05};`. Define each variable as close as possible to where it is first used, not at the top of the function as in old C.

2.5 `auto`: let the compiler deduce the type

```
auto i = 42;          // int
auto d = 3.14;        // double
auto s = std::string{"hi"};   // std::string
auto it = v.begin();  // std::vector<int>::iterator, much shorter to write
```

`auto` takes the type from the initialiser, so the variable must be initialised. Note that `auto` drops top-level `const` and references: `auto x = someConstRef;` makes a plain copy. Write `const auto&` when you want a read-only reference. Use `auto` when the type is obvious or very long; write the type out when it helps the reader.

2.6 Type aliases

```
using Price  = double;                          // modern form
typedef double Price;                           // old C form, same meaning
using Matrix = std::vector<std::vector<double>>;
```

Aliases make code self-documenting (`Price` says more than `double`) and give you one place to change a type later.

2.7 Enumerations: prefer `enum class`

```
enum Color { red, green, blue };            // C-style (unscoped)
enum class Side { buy, sell };              // scoped enumeration

Side s = Side::buy;                         // must qualify with Side::
// int n = s;                               // error: no silent conversion to int
int n = static_cast<int>(s);                // explicit conversion is fine
```

Unscoped enums leak their names into the surrounding scope (you cannot then have another enum with `red`) and convert silently to integers. Scoped enums fix both problems.

2.8 Structs

A `struct` groups related data. In C++ you no longer need to write `struct` before the type name, and members can have default values.

```
struct Point {
    double x{0.0};
    double y{0.0};
};

Point p{3.0, 4.0};                // aggregate initialisation
Point origin{};                   // x = 0, y = 0
std::cout << p.x << ", " << p.y << '\n';
```

In C++ a `struct` can also have member functions, constructors and everything a class has. The only difference is the default access level (Chapter 7).

2.9 Strings

```
#include <string>
std::string s = "Hello";
s += ", world";                         // append
std::cout << s.size() << '\n';          // 12
std::cout << s.substr(0, 5) << '\n';    // "Hello"
if (s.find("world") != std::string::npos) { /* found */ }
```

`std::string` owns its characters and frees them automatically. For read-only string parameters, C++17 added `std::string_view`: a lightweight, non-owning *view* of characters that avoids copying.

```
#include <string_view>
void greet(std::string_view name) {   // accepts std::string, "literals", char*
    std::cout << "Hi " << name << '\n';
}
```

A string_view must not outlive what it views

`std::string_view` does not own its characters. If the string it points into is destroyed, the view dangles. Use `string_view` for function parameters; be careful storing one in a variable or a class.

2.10 Range-based for loops

```
std::vector<int> v{1, 2, 3};
for (int x : v) std::cout << x << ' ';          // copy of each element
for (int& x : v) x *= 2;                         // modify in place
for (const auto& s : names) std::cout << s;      // read without copying
```

Use `const auto&` for anything bigger than a few bytes (strings, structs), so each element is not copied.

Chapter 3

Constants: `const`, `constexpr` and `inline`

Few keywords confuse beginners more than `const` and `constexpr`. They are related but answer different questions. This chapter takes them slowly.

Key idea

`const` means "**this value will not change after it is set**". The value may be known only at runtime.
`constexpr` means "**this value is known at compile time**" (and is therefore also constant).

3.1 `const` variables

```
const double gravity{9.81};       // never changes
const int maxUsers{100};

int age{};
std::cin >> age;
const int ageNextYear{age + 1};   // fine: const, but only known at runtime

// gravity = 10.0;                // compile error: cannot modify a const
// const int z;                   // compile error: const must be initialised
```

A `const` variable must be initialised when created, because you can never assign to it later. Making values `const` wherever possible helps in three ways: the compiler catches accidental changes, readers know the value is fixed, and the optimiser has more freedom.

3.1.1 Why not `#define`?

In C, constants were often macros: `#define MAX_USERS 100`. The preprocessor replaces the text before the compiler ever sees it. That causes problems:

- A macro has no type, so the compiler cannot check how you use it.
- A macro ignores scope. Once defined, it replaces that word everywhere below it in the file, which can cause baffling errors far from the macro itself.
- A macro has no name in the debugger; you only see the number.

Best practice

Use `constexpr` (or `const`) variables instead of `#define` for constants. Keep macros for the few jobs only the preprocessor can do, such as header guards and conditional compilation.

3.2 Compile time versus runtime

To understand `constexpr`, separate two moments in a program's life:

- **Compile time**: when the compiler translates your code. Anything computed here costs nothing when the program runs.
- **Runtime**: when the program executes, after reading input, files and so on.

A *constant expression* is an expression the compiler can evaluate completely at compile time: `2 + 3`, `sizeof(int)`, or a `constexpr` variable. Some places in the language *require* a constant expression: the size of a `std::array`, a template argument, a `case` label, or a `static_assert` condition.

3.3 `constexpr` variables

```
constexpr double pi{3.14159265358979};
constexpr int    daysInWeek{7};
constexpr int    hoursInWeek{daysInWeek * 24};   // computed by the compiler

std::array<int, daysInWeek> sales{};             // OK: size must be compile-time

int n{};
std::cin >> n;
// constexpr int m{n * 2};    // error: n is only known at runtime
const int m{n * 2};           // fine: const allows runtime values
```

	`const`	`constexpr`
Can it change after initialisation?	No	No
Known at compile time?	Not necessarily	Always
Usable as an array size or template argument?	Only if its initialiser is a constant expression	Always
Typical use	Values fixed once computed; parameters; member functions	True constants: physical constants, sizes, configuration

Best practice

If a constant's value can be known at compile time, make it `constexpr`. Otherwise make it `const`.

3.4 `constexpr` functions

A `constexpr` function *can* be evaluated at compile time when its arguments are constant expressions. With runtime arguments, it simply runs at runtime like a normal function. You write the logic once and get both behaviours.

```
constexpr long long factorial(int n) {
    long long result{1};
    for (int i{2}; i <= n; ++i) result *= i;
    return result;
}

constexpr long long f10{factorial(10)};   // computed while compiling: 3628800
static_assert(factorial(5) == 120);       // checked while compiling
```

```
int k{};
std::cin >> k;
std::cout << factorial(k) << '\n';        // computed at runtime
```

Since C++20, `constexpr` functions can use loops, local variables, `if`, `switch`, and even `std::vector` and `std::string` internally (as long as nothing allocated escapes to runtime). They cannot do things that only make sense at runtime, such as reading input.

3.4.1 `consteval` and `constinit` (C++20)

- `consteval` marks an *immediate function*: it **must** run at compile time. Calling it with a runtime value is a compile error. Use it when a runtime call would always be a bug.
- `constinit` on a global or static variable guarantees it is initialised at compile time, but unlike `constexpr` the variable can still be changed later. It is a niche tool for avoiding problems with the order in which global variables are initialised.

```
consteval int square(int x) { return x * x; }
constexpr int a{square(4)};   // OK: 16 at compile time
// int b{square(k)};          // error: k is not a constant expression
```

3.5 `static_assert`: checks while compiling

```
static_assert(sizeof(int) == 4, "This code assumes 32-bit int");
static_assert(hoursInWeek == 168);
```

If the condition is false, the program does not compile. Compare this with `assert(cond)` from `<cassert>`, which checks at runtime (and is switched off when `NDEBUG` is defined, as in most release builds).

3.6 `if constexpr`

Inside templates you sometimes want different code for different types. `if constexpr` chooses a branch at compile time and throws away the other branch entirely:

```
#include <type_traits>
template <typename T>
void describe(const T& value) {
    if constexpr (std::is_integral_v<T>)
        std::cout << value << " is an integer\n";
    else
        std::cout << value << " is something else\n";
}
```

You will meet templates properly in Chapter 16.

3.7 Inline functions

3.7.1 The original meaning: avoid function-call overhead

Calling a function has a small cost: passing arguments, jumping, returning. For tiny functions, C programmers often used macros instead. Macros are dangerous, though:

```
#define SQUARE(x) x * x
int a = SQUARE(2 + 3);   // expands to 2 + 3 * 2 + 3 = 11, not 25!

#define SQUARE2(x) ((x) * (x))
int i = 3;
int b = SQUARE2(i++);    // expands to ((i++) * (i++)): undefined behaviour
```

An `inline` function behaves exactly like a normal function (arguments are evaluated once, types are checked), while inviting the compiler to paste the body at the call site instead of making a call:

```
inline int square(int x) { return x * x; }
int a = square(2 + 3);   // 25, as expected
```

3.7.2 The modern meaning: "may be defined in more than one file"

Today, compilers decide on their own which functions to expand inline; they mostly ignore the hint. The keyword `inline` matters for a different reason: it relaxes the One Definition Rule. An `inline` function may be defined in a header that many `.cpp` files include, as long as every definition is identical. The linker keeps one copy.

```
// utils.h
#pragma once
inline double clamp01(double x) {        // safe to define in a header
    return x < 0 ? 0 : (x > 1 ? 1 : x);
}
```

Several things are implicitly inline: member functions defined inside a class body, `constexpr` functions, and function templates. That is why they can live in headers.

3.7.3 Inline variables (C++17)

The same idea applies to global constants shared across files:

```
// constants.h
#pragma once
namespace constants {
    inline constexpr double pi{3.14159265358979};
    inline constexpr double e{2.71828182845905};
}
```

Every file that includes this header uses the same single `constants::pi`.

Key idea

Write `inline` to put a function or variable definition in a header. Do not write it to try to make code faster; the optimiser decides that better than you can.

The other uses of `const` (with pointers, references, member functions and objects) appear in the chapters where those ideas are introduced, and Section 8.5 brings them together.

Chapter 4

References and Pointers

Pointers and references both let you refer to an object without copying it. Getting comfortable with them is the single biggest step from beginner to intermediate C++.

4.1 Every object lives at an address

Each variable occupies some bytes in memory, and those bytes have an address. The address-of operator `&` gives it to you.

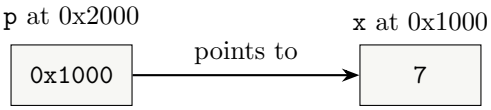
```
int x{42};
std::cout << x  << '\n';   // 42
std::cout << &x << '\n';   // something like 0x7ffd5c3a1b2c
```

4.2 Pointers

A pointer is a variable that *stores an address*.

```
int x{42};
int* p{&x};          // p holds the address of x
std::cout << *p;     // 42   (* "dereferences": go to the address, read the value)
*p = 7;              // changes x through the pointer
std::cout << x;      // 7

int* q{nullptr};     // a pointer that points at nothing
if (q) { /* safe to use *q */ }
```



The symbols `*` and `&` each mean two different things depending on where they appear:

	In a declaration (a type)	In an expression
`*`	`int* p` — "p is a pointer to int"	`*p` — "the object p points to"
`&`	`int& r` — "r is a reference to int"	`&x` — "the address of x"

Dereferencing null or dangling pointers

Dereferencing `nullptr`, an uninitialised pointer, or a pointer to an object that no longer exists is undefined behaviour. Initialise every pointer, either to a real address or to `nullptr`, and check before dereferencing when it might be null. Never use `NULL` or `0` for pointers in C++: `nullptr` has its own type and cannot be confused with the integer zero.

4.2.1 Pointers and arrays

A C-style array name converts ("decays") to a pointer to its first element. Pointer arithmetic moves by whole elements, not bytes: `p + 1` points to the next `int`, four bytes further on.

```
int arr[3]{10, 20, 30};
int* p{arr};                // same as &arr[0]
std::cout << *(p + 2);      // 30, same as arr[2] and p[2]
```

Once an array has decayed to a pointer, its length is lost, which is why C functions always take a separate length argument. In C++, prefer `std::array` or `std::vector`, which know their size.

4.3 References

A reference is an **alias**: another name for an existing object.

```
int x{42};
int& r{x};      // r is another name for x
r = 7;          // x is now 7
std::cout << x; // 7
```

	Pointer	**Reference**
Can be null	Yes (`nullptr`)	No: must be bound to an object
Must be initialised	No (but should be)	Yes
Can be changed to refer elsewhere	Yes	No: bound for life
Syntax to reach the object	`*p`, `p->member`	Just use it: `r`, `r.member`
Has its own address	Yes	Conceptually no; it is just a name

Key idea

Use a reference when something must always refer to a valid object and never needs to be reseated. Use a pointer when "nothing" (`nullptr`) is a meaningful value, or when what it points to must change.

4.4 Passing arguments to functions

This is where references earn their keep.

```
void byValue(int x)        { x = 100; }  // works on a copy; caller unchanged
void byReference(int& x)   { x = 100; }  // works on the caller's variable
void byAddress(int* x)     { if (x) *x = 100; }  // C style; caller passes &a

int a{1};
byValue(a);      // a == 1
byReference(a);  // a == 100
byAddress(&a);   // a == 100
```

4.4.1 Pass by const reference: the everyday default

Copying a large object (a `std::string`, a vector of a million numbers) is expensive. A *const reference* gives you the original without copying it, and promises not to modify it.

```
double average(const std::vector<double>& values) {   // no copy, read-only
    double sum{0.0};
    for (double v : values) sum += v;
    return values.empty() ? 0.0 : sum / values.size();
}
```

A const reference can also bind to temporaries, so `average({1.0, 2.0, 3.0})` works. A non-const reference cannot.

Choosing how to pass a parameter

- Cheap-to-copy types (`int`, `double`, `char`, pointers, `std::string_view`): pass by value.
- Everything else that you only read: pass by `const T&`.
- Something you must modify: pass by `T&` (and name the function so this is obvious).
- Something optional (may be absent): pass a pointer and allow `nullptr`, or use `std::optional`.

4.5 Returning references, and dangling

Returning a reference avoids a copy, but the object referred to must outlive the function call.

```
int& bad() {
    int local{5};
    return local;          // DANGER: local dies when bad() returns
}                          // the caller gets a dangling reference

const std::string& firstName(const std::vector<std::string>& names) {
    return names[0];       // fine: the vector belongs to the caller and outlives
    this call
}
```

Never return a reference or pointer to a local variable

Local variables are destroyed when the function returns. A reference or pointer to them *dangles*: it refers to memory that may already hold something else. Compilers usually warn about the simple case, so read warnings.

4.6 `const` with pointers: read the declaration right to left

With pointers there are two things that can be constant: the pointer itself, and the object it points to. The trick is to read the type from right to left.

```
int x{1}, y{2};

const int* p1{&x};         // "p1 is a pointer to a const int"
// *p1 = 5;                // error: cannot change x through p1
p1 = &y;                   // OK: p1 itself may point elsewhere
```

```
int* const p2{&x};         // "p2 is a const pointer to int"
*p2 = 5;                   // OK: may change x
// p2 = &y;                // error: p2 cannot be reseated

const int* const p3{&x};   // "p3 is a const pointer to a const int"
// neither *p3 = 5 nor p3 = &y is allowed
```

Declaration	Change the pointee?	Reseat the pointer?
`int* p`	yes	yes
`const int* p` (same as `int const* p`)	no	yes
`int* const p`	yes	no
`const int* const p`	no	no

References cannot be reseated anyway, so for references only the pointee matters: `const int& r` is a read-only alias.

4.7 lvalues and rvalues (a first look)

Every expression is either an *lvalue* or an *rvalue*. Roughly:

- An **lvalue** has a name or an identity that persists: `x`, `arr[2]`, `*p`. You can take its address.
- An **rvalue** is a temporary value that disappears at the end of the expression: `42`, `x + 1`, the result of `makeString()`.

A plain reference `T&` binds only to lvalues. A `const T&` binds to both. C++11 added *rvalue references* `T&&`, which bind only to temporaries. They are the foundation of move semantics, explained carefully in Chapter 18.

Chapter 5

Functions: Default Arguments, Overloading and More

5.1 Default arguments

A parameter can have a default value that is used when the caller leaves it out.

```
void printLine(char ch = '-', int width = 40) {
    std::cout << std::string(width, ch) << '\n';
}

printLine();          // 40 dashes
printLine('=');       // 40 equals signs
printLine('*', 10);   // 10 stars
```

Rules worth remembering:

- Defaults must be at the *end* of the parameter list. Once one parameter has a default, all parameters to its right need one too. You cannot skip a middle argument: `printLine(, 10)` is not valid.
- Put the default in the declaration (usually in the header), *not* again in the definition.

```
// shapes.h
double area(double width, double height = 1.0);
// shapes.cpp
double area(double width, double height) { return width * height; }  // no default
    here
```

5.2 Function overloading

Several functions can share a name if their parameter lists differ. The compiler picks the right one from the arguments.

```
int    add(int a, int b)          { return a + b; }
double add(double a, double b)    { return a + b; }
std::string add(const std::string& a, const std::string& b) { return a + b; }

add(1, 2);              // calls add(int, int)
add(1.5, 2.5);          // calls add(double, double)
add(std::string{"a"}, std::string{"b"});   // calls the string version
```

Overloads must differ in the number or types of parameters. A different *return type alone* is not enough, because the compiler chooses an overload from the call, and the call does not mention the return type.

5.2.1 How the compiler chooses (overload resolution)

The compiler ranks each candidate by how well the arguments match, roughly in this order:

1. **Exact match** (including trivial changes such as adding `const`).
2. **Promotion**: for example `char` or `short` to `int`, `float` to `double`.
3. **Standard conversion**: for example `int` to `double`, `double` to `int`.
4. **User-defined conversion**: through a constructor or conversion operator you wrote.

The best-ranked candidate wins. If two candidates tie, the call is *ambiguous* and fails to compile:

```
void f(int);
void f(double);
f('a');      // OK: char -> int is a promotion, beats char -> double
f(5L);       // ERROR: long -> int and long -> double are both conversions (tie)
```

Overloading plus default arguments

`void g(int a);` and `void g(int a, int b = 0);` together make `g(5)` ambiguous. Choose one mechanism or the other for a given function name.

5.3 Deleting functions

You can forbid particular calls by marking an overload `= delete`:

```
void setAge(int years);
void setAge(double) = delete;   // setAge(25.7) is now a compile error
void setAge(char)   = delete;   // setAge('a') too
```

The same syntax is used to make a class non-copyable (Chapter 8).

5.4 Returning results

5.4.1 Multiple values

Return a small struct, which is clearer than output parameters:

```
struct MinMax { int min; int max; };

MinMax findMinMax(const std::vector<int>& v) {
    auto [lo, hi] = std::minmax_element(v.begin(), v.end());
    return {*lo, *hi};
}

auto [mn, mx] = findMinMax({4, 1, 9});   // structured bindings (C++17)
```

5.4.2 Maybe a value: `std::optional`

```
#include <optional>
std::optional<int> parseAge(const std::string& s) {
    try { return std::stoi(s); }
    catch (...) { return std::nullopt; }   // "no value"
}
```

```
if (auto age = parseAge(input)) std::cout << *age;
else std::cout << "invalid\n";
```

5.4.3 Do not ignore the result: `[[nodiscard]]`

```
[[nodiscard]] bool saveFile(const std::string& path);
saveFile("a.txt");   // warning: ignoring a return value marked nodiscard
```

5.5 Function pointers

Functions also have addresses, so you can store a function in a variable and call it later.

```
int add(int a, int b) { return a + b; }
int mul(int a, int b) { return a * b; }

int (*op)(int, int){&add};    // op points to a function taking (int,int) returning
    int
std::cout << op(2, 3);        // 5
op = &mul;
std::cout << op(2, 3);        // 6

using BinaryOp = int (*)(int, int);   // a type alias makes this readable
int apply(BinaryOp f, int a, int b) { return f(a, b); }
```

Function pointers are how C passes behaviour to other code (like `qsort`'s comparison function). Modern C++ usually uses lambdas or `std::function` instead (Chapter 17).

5.6 Lambdas in one page

A lambda is a function you can write inline, right where it is needed.

```
std::vector<int> v{5, 2, 8, 1};
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });  // descending

int threshold{4};
auto bigOnes = std::count_if(v.begin(), v.end(),
                             [threshold](int x) { return x > threshold; });
```

The parts: `[capture list]` says which surrounding variables the lambda may use; `(parameters)` and { `body` } work as in a normal function; the return type is deduced. Chapter 17 explains captures and what a lambda really is.

5.7 The call stack, and why recursion has limits

Each function call gets a *stack frame* holding its parameters and local variables. The frame is created when the function is called and removed when it returns, which is why local variables vanish. The stack is small (often around 8 MB), so very deep recursion, or a huge local array such as `int big[10000000];`, overflows it and crashes. Large data belongs on the heap, usually inside a `std::vector` (Chapter 6).

Chapter 6

Dynamic Memory Management

6.1 Where objects live

A running program uses several memory regions:

Region	What lives there	Lifetime
Static storage	Global variables, `static` variables	Whole program
Stack	Local variables and function parameters	Until the enclosing block ends
Heap (free store)	Objects created with `new`	Until you `delete` them

The stack is fast and automatic but small, and its objects die when their scope ends. The heap is large and flexible: an object lives as long as you want, and its size can be decided at runtime. The price is that *someone must free it*.

6.2 `new` and `delete`

```
int* p{new int{42}};      // allocate one int on the heap, initialise to 42
std::cout << *p;
delete p;                 // free it
p = nullptr;              // good habit: p no longer dangles

int n{};
std::cin >> n;
double* arr{new double[n]{}};   // array of n doubles, all zero
arr[0] = 1.5;
delete[] arr;                   // arrays need delete[], not delete
```

6.2.1 `new` versus `malloc`

	`malloc`/`free` (C)	`new`/`delete` (C++)
Returns	`void*`; you must cast	A correctly typed pointer
Size	You compute bytes: `malloc(n * sizeof(T))`	Computed for you
Constructors and destructors	Not called	Called
On failure	Returns `NULL`	Throws `std::bad_alloc`

Never mix them: memory from `malloc` must go to `free`, from `new` to `delete`, and from `new[]` to `delete[]`.

6.3 The classic bugs

Manual memory management goes wrong in four standard ways. All four compile without complaint.

```
// 1. Memory leak: the memory is never freed
void leak() {
    int* p{new int[1000]};
    if (somethingFailed()) return;   // early return: delete[] is skipped
    delete[] p;
}

// 2. Dangling pointer: using memory after freeing it
int* p{new int{5}};
delete p;
std::cout << *p;        // undefined behaviour

// 3. Double delete
int* a{new int{1}};
int* b{a};
delete a;
delete b;               // undefined behaviour: same memory freed twice

// 4. Mismatched delete
int* arr{new int[10]};
delete arr;             // should be delete[] arr: undefined behaviour
```

Exceptions make leaks even easier: if anything between `new` and `delete` throws, the `delete` never runs.

The modern answer: let objects own memory (RAII)

Instead of pairing `new` and `delete` by hand, put the resource inside an object whose *destructor* frees it. When the object goes out of scope, for any reason (normal return, early return or exception), its destructor runs automatically. This idea is called **RAII**, "Resource Acquisition Is Initialisation", and it is the most important idiom in C++. `std::vector`, `std::string`, `std::unique_ptr` and `std::shared_ptr` all work this way.

```
#include <vector>
#include <memory>

void noLeak(int n) {
    std::vector<double> data(n);                    // heap array, freed automatically
    auto obj{std::make_unique<Widget>()};           // heap object, freed
    automatically
    if (somethingFailed()) return;                  // still no leak
}
```

Best practice

In modern code you should almost never write `new` or `delete` yourself. Use containers for collections and `std::make_unique` for single heap objects. Learn `new`/`delete` anyway: you need them to understand how classes manage resources (Chapter 8) and how smart pointers work (Chapter 18).

Chapter 7

Classes and Objects

7.1 Why object-oriented programming?

In C, data (a `struct`) and the functions that work on it are separate, and nothing stops any code anywhere from putting the data into a broken state. Object-oriented programming (OOP) bundles data with the operations that keep it valid. Four ideas run through it:

- **Encapsulation**: keep the data private, and expose a small set of operations that keep it correct.
- **Abstraction**: users of a class think about *what* it does, not *how*.
- **Inheritance**: build new classes from existing ones (Chapter 12).
- **Polymorphism**: treat different types through one common interface (Chapter 13).

A **class** is a blueprint, a user-defined type. An **object** is one instance of that type, with its own copy of the data members.

7.2 A first class

```
class BankAccount {
public:                                  // accessible from anywhere
    void deposit(double amount) {
        if (amount > 0) balance_ += amount;
    }
    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance_) return false;
        balance_ -= amount;
        return true;
    }
    double balance() const { return balance_; }   // const: does not modify the object

private:                                 // accessible only inside the class
    double balance_{0.0};                // data member with a default value
};

int main() {
    BankAccount acc;              // an object
    acc.deposit(500);
    acc.withdraw(200);
    std::cout << acc.balance();   // 300
    // acc.balance_ = 1e9;        // error: balance_ is private
}
```

Because `balance_` is private, the only way to change it is through `deposit` and `withdraw`, which both protect the rule "the balance never goes negative". A rule like this that must always hold is called a *class invariant*. Encapsulation exists to protect invariants.

(The trailing underscore in `balance_` is a common naming convention for private data members. Others use `m_balance`. Pick one and be consistent.)

7.3 Access specifiers

Specifier	Who can access the member
`public`	Everyone
`protected`	The class itself, its friends, and classes derived from it (Chapter 12)
`private`	Only the class itself and its friends

Access control works per *class*, not per object: a member function of `BankAccount` may read the private members of *another* `BankAccount` object. That is what makes functions like a copy constructor possible.

7.3.1 `class` versus `struct`

The only language difference: members of a `struct` are `public` by default, and members of a `class` are `private` by default. By convention, use `struct` for simple bundles of data with no invariants (a point, a configuration record) and `class` for types that protect invariants.

7.4 Member functions defined outside the class

For larger classes, the class definition goes in a header and the function bodies in a `.cpp` file. The *scope resolution operator* `::` says which class a function belongs to.

```
// BankAccount.h
#pragma once
class BankAccount {
public:
    void deposit(double amount);
    double balance() const;
private:
    double balance_{0.0};
};

// BankAccount.cpp
#include "BankAccount.h"
void BankAccount::deposit(double amount) {
    if (amount > 0) balance_ += amount;
}
double BankAccount::balance() const { return balance_; }
```

Functions defined *inside* the class body are implicitly `inline`, so it is safe to keep short ones in the header.

7.5 The hidden `this` pointer

How does `deposit` know *which* account's balance to change? Every non-static member function receives a hidden extra parameter, `this`, a pointer to the object it was called on. The compiler turns

```
acc.deposit(500);
```

into something like `BankAccount::deposit(&acc, 500)`, and inside the function `balance_` really means `this->balance_`. You rarely need to write `this` explicitly, but it is useful in two cases:

```
class Point {
public:
    void setX(double x) { this->x = x; }   // 1. a parameter has the same name as a
    member

    Point& moveBy(double dx, double dy) {  // 2. returning *this enables chaining
        x += dx; y += dy;
        return *this;
    }
private:
    double x{}, y{};
};

Point p;
p.moveBy(1, 2).moveBy(3, 4);               // calls chained on the same object
```

7.6 Const objects and const member functions

A `const` object may only call member functions that promise not to modify it. You make that promise by writing `const` after the parameter list.

```
class Temperature {
public:
    double celsius() const { return c_; }          // const member function
    void   set(double c)   { c_ = c; }             // non-const
private:
    double c_{0.0};
};

const Temperature t{};      // const object
t.celsius();                // OK
// t.set(30);               // error: set() is not const

void show(const Temperature& temp) {   // const reference: same rules
    std::cout << temp.celsius();       // OK only because celsius() is const
}
```

Inside a const member function, `this` has type `const Temperature*`, so every data member behaves as `const`. Trying to modify one is a compile error.

Best practice

Mark every member function that does not change the object as `const`. If you forget, your class becomes unusable through const references, which is how objects are usually passed.

7.6.1 `mutable`

Occasionally a member changes for internal reasons that do not change the object's *observable* state, such as a cache or a counter used for debugging. Marking it `mutable` allows a const member

function to modify it.

```
class Report {
public:
    const std::string& text() const {
        if (!cached_) { cache_ = buildExpensiveText(); cached_ = true; }
        return cache_;
    }
private:
    std::string buildExpensiveText() const;
    mutable std::string cache_;
    mutable bool cached_{false};
};
```

Use `mutable` sparingly: it is an exception to a promise, so it should be obviously justified.

7.7 Getters and setters, used wisely

Access functions (`getX`, `setX`) are fine, but writing a public getter and setter for every private member undoes encapsulation: the data is effectively public again. Design the public interface around what users need to *do* (`deposit`, `withdraw`), not around the data you store.

Chapter 8

Constructors, Destructors and Copying

8.1 Constructors

A constructor is a special member function that runs when an object is created. Its job is to put the object into a valid state, establishing its invariants from the very first moment. It has the same name as the class and no return type.

```
class Fraction {
public:
    Fraction() : num_{0}, den_{1} {}                  // default constructor
    Fraction(int n, int d) : num_{n}, den_{d} {       // parameterised constructor
        if (d == 0) throw std::invalid_argument("zero denominator");
    }
private:
    int num_;
    int den_;
};

Fraction a;           // calls Fraction()
Fraction b{3, 4};     // calls Fraction(int, int)
```

8.1.1 The member initialiser list

The part after the colon, `: num_{n}, den_{d}`, is the *member initialiser list*. It *initialises* members as they are created. Assigning inside the body instead would first default-construct each member and then overwrite it.

The initialiser list is *required* for:

- `const` data members and reference members (they cannot be assigned later);
- members of a class type that has no default constructor;
- base classes that need constructor arguments (Chapter 12).

Initialisation order follows declaration order

Members are always initialised in the order they are *declared in the class*, not the order in the initialiser list. If `den_` were declared before `num_` and initialised as `den_{num_ * 2}`, it would read `num_` before `num_` was set. Keep the list in declaration order; `-Wall` warns you when it is not.

8.1.2 Default member initialisers and the default constructor

```
class Config {
public:
    Config() = default;               // ask the compiler to generate it
    explicit Config(int threads) : threads_{threads} {}
private:
```

```
    int  threads_{4};                 // used when a constructor does not set it
    bool verbose_{false};
};
```

If you write *no* constructors at all, the compiler generates a default constructor for you. As soon as you write any constructor, it no longer does; write `= default` to get it back.

8.1.3 Delegating constructors

One constructor can call another to avoid repeating code:

```
class Rectangle {
public:
    Rectangle(double w, double h) : w_{w}, h_{h} {}
    explicit Rectangle(double side) : Rectangle(side, side) {}   // a square
private:
    double w_, h_;
};
```

8.1.4 `explicit`: stop surprise conversions

A constructor callable with one argument doubles as an implicit conversion:

```
class Meters {
public:
    Meters(double v) : v_{v} {}       // not explicit
private:
    double v_;
};
void run(Meters distance);
run(5.0);                             // compiles: 5.0 silently becomes Meters{5.0}
```

That is sometimes convenient, but often hides bugs. Marking the constructor `explicit` requires the caller to write `run(Meters{5.0})`.

Best practice

Make single-argument constructors `explicit` unless you specifically want implicit conversion.

8.2 Destructors

A destructor runs automatically when an object's lifetime ends. Its name is the class name with a tilde, it takes no parameters, and there is exactly one per class. Its job is to release whatever the object owns.

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : size_{n}, data_{new int[n]{}} {
        std::cout << "acquired " << n << " ints\n";
    }
    ~Buffer() {
        delete[] data_;
        std::cout << "released\n";
    }
```

```
private:
    std::size_t size_;
    int* data_;
};

void work() {
    Buffer b{100};       // "acquired 100 ints"
    // ... use b ...
}                        // b goes out of scope: "released"
```

This is RAII in action (Chapter 6): the constructor acquires, the destructor releases, and the language guarantees the destructor runs.

8.3 Object lifetime

Kind of object	Constructed	Destroyed
Local (automatic)	When execution reaches its definition	At the end of its enclosing block
Global or `static`	Before `main` (local statics: on first use)	After `main` returns
Dynamic (`new`)	At `new`	At `delete` (never, if you forget)
Temporary	In an expression	At the end of the full expression
Data member	Before the body of its owner's constructor	After the body of its owner's destructor

Key idea

Objects in the same scope are destroyed in the *reverse* order of their construction. Later objects may depend on earlier ones, so they must go first. The same rule applies to data members and to base classes.

8.4 Copying objects

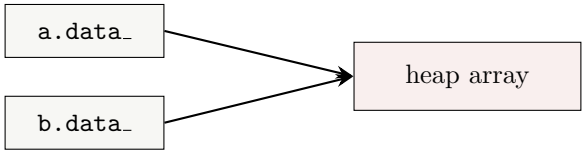
Two special member functions control copying:

```
Buffer a{10};
Buffer b{a};        // copy CONSTRUCTOR: creates b as a copy of a
Buffer c{5};
c = a;              // copy ASSIGNMENT: replaces c's contents with a copy of a's
```

The copy constructor creates a *new* object. Copy assignment overwrites an *existing* one, which must first clean up what it already owns. If you do not write them, the compiler generates both, copying each member one by one.

8.4.1 Shallow versus deep copy: the classic disaster

For `Buffer`, member-by-member copying copies the *pointer* `data_`, not the array it points to. Both objects now point at the same heap memory:



When `b` is destroyed it deletes the array; then `a`'s destructor deletes the same array again (a double delete), and any use of `a` in between reads freed memory. This is a *shallow copy*. The fix is a *deep copy*: allocate new memory and copy the contents.

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : size_{n}, data_{new int[n]{}} {}
    ~Buffer() { delete[] data_; }

    // Copy constructor: allocate fresh memory, copy contents
    Buffer(const Buffer& other)
        : size_{other.size_}, data_{new int[other.size_]} {
        std::copy(other.data_, other.data_ + size_, data_);
    }

    // Copy assignment: copy-and-swap idiom
    Buffer& operator=(Buffer other) {   // note: other is taken BY VALUE (a copy)
        swap(other);                    // take its contents, give it ours
        return *this;                   // other's destructor frees our old memory
    }

    void swap(Buffer& other) noexcept {
        std::swap(size_, other.size_);
        std::swap(data_, other.data_);
    }
private:
    std::size_t size_;
    int* data_;
};
```

The copy-and-swap idiom deserves a moment. The parameter `other` is a fresh copy made by the copy constructor. We swap our contents with it; now we hold the new data and `other` holds our old data, which `other`'s destructor frees when the function returns. It handles self-assignment (`a = a`) correctly, and if making the copy throws, our object is untouched.

8.4.2 The Rule of Three, Five and Zero

Key idea

Rule of Three: if a class needs a custom destructor, copy constructor *or* copy assignment operator, it almost certainly needs all three, because it manages a resource by hand.

Rule of Five (C++11): add the move constructor and move assignment operator (Chapter 18).

Rule of Zero: best of all, write *none* of them. Hold resources in members that already manage themselves (`std::vector`, `std::string`, `std::unique_ptr`), and the compiler-generated versions do the right thing.

```
class Buffer {                      // Rule of Zero version: correct, and 3 lines long
public:
    explicit Buffer(std::size_t n) : data_(n) {}
private:
```

```
    std::vector<int> data_;         // copies deeply and frees itself
};
```

8.4.3 Disabling copying

Some objects should never be copied: a file handle, a mutex, a network connection.

```
class FileHandle {
public:
    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;
    // ...
};
```

8.4.4 Copy elision

Returning a local object by value does not usually copy it. Since C++17 the compiler is *required* to construct a returned temporary directly in the caller's variable, and it usually does the same for named local variables. So returning a `std::vector` by value is cheap; do not contort your code to avoid it.

8.5 Const-ness: the whole picture

`const` appears in many places. Each one is a promise the compiler enforces:

Where	Meaning
`const int x{5};`	This variable will not change
`constexpr int x{5};`	This value is known at compile time (and is constant)
`const int* p`	Cannot change the pointed-to object through `p`
`int* const p`	`p` itself cannot be reseated
`void f(const std::string& s)`	`f` reads `s` without copying or changing it
`int get() const;`	This member function does not modify the object
`const Widget w;`	Only `const` member functions may be called on `w`
`const Widget& w = obj;`	A read-only view of `obj`
`mutable int cache_;`	This member may change even in const member functions
`const T& get() const;`	Returns read-only access to an internal member

Const-correctness

Make things `const` by default and remove `const` only when you need to modify. Code written this way documents itself, catches mistakes at compile time, and works smoothly with the many standard-library functions that hand you const references.

You can overload a member function on `const`-ness, which is how containers give read-only access to const objects and writable access to non-const ones:

```
class Row {
public:
    double&       operator[](std::size_t i)       { return v_[i]; }  // non-const
    object
```

```
    const double& operator[](std::size_t i) const { return v_[i]; }  // const object
private:
    std::vector<double> v_;
};
```

Chapter 9

Static Members and Friends

9.1 The many meanings of `static`

`static` is one of C++'s most overloaded keywords. Its meaning depends on where it appears:

Where	Meaning
Local variable inside a function	Created once, keeps its value between calls, lives until the program ends
Global variable or function at file scope	Visible only inside this `.cpp` file (internal linkage)
Data member of a class	One shared copy for the whole class, not one per object
Member function of a class	Belongs to the class; has no `this` pointer

9.1.1 Static local variables

```
int nextId() {
    static int counter{0};   // initialised only the first time
    return ++counter;
}
nextId();   // 1
nextId();   // 2
```

9.2 Static data members

A normal data member exists once per object. A `static` data member exists once for the entire class, shared by every object, and it exists even when no objects do.

```
class Order {
public:
    Order() : id_{++count_} { ++live_; }
    ~Order() { --live_; }
    static int count() { return count_; }

private:
    int id_;
    static inline int count_{0};   // C++17: define and initialise inside the class
    static inline int live_{0};
};

Order a, b, c;
std::cout << Order::count();   // 3, accessed through the class name
```

Before C++17, a static data member was only *declared* in the class, and had to be *defined* in exactly one `.cpp` file:

```
// Order.h
class Order { static int count_; /* ... */ };
```

```
// Order.cpp
int Order::count_ = 0;          // the single definition
```

`static inline` (C++17) removes that need. A `static constexpr` member is also implicitly inline.

9.3 Static member functions

A static member function belongs to the class rather than to any object, so it has no `this` pointer and cannot touch non-static members directly. Call it with the class name: `Order::count()`.

Typical uses are factory functions and helpers related to the class:

```
class Temperature {
public:
    static Temperature fromFahrenheit(double f) { return Temperature{(f - 32) * 5 /
    9}; }
    static Temperature fromCelsius(double c)    { return Temperature{c}; }
private:
    explicit Temperature(double c) : c_{c} {}   // private: use the named factories
    double c_;
};

auto t{Temperature::fromFahrenheit(98.6)};
```

Named factory functions read better than several constructors whose parameters are all `double`.

9.4 Friend functions and friend classes

Sometimes a function that is *not* a member still needs access to a class's private members. Declaring it a `friend` grants that access.

```
class Vec2 {
public:
    Vec2(double x, double y) : x_{x}, y_{y} {}
    friend double dot(const Vec2& a, const Vec2& b);            // friend function
    friend std::ostream& operator<<(std::ostream& os, const Vec2& v);
private:
    double x_, y_;
};

double dot(const Vec2& a, const Vec2& b) {        // not a member: no Vec2:: prefix
    return a.x_ * b.x_ + a.y_ * b.y_;              // but may read private members
}
```

A whole class can also be a friend: `friend class Matrix;` inside `Vec2` lets every member function of `Matrix` access `Vec2`'s private members.

Facts about friendship:

- Friendship is *granted* by the class, never taken. Only `Vec2` can say who its friends are.
- It is not mutual: if `A` declares `B` a friend, `A` gets no access to `B`.
- It is not inherited and not transitive: a friend's friend, or a friend's derived class, has no access.

Best practice

Friends are the standard way to write operators such as `<<` that need private data but cannot be members (Chapter 10). Beyond that, use friendship rarely: every friend is another piece of code that can break your invariants.

Chapter 10

Operator Overloading

Operator overloading lets your own types use natural syntax: `a + b` for complex numbers, `m[i][j]` for a matrix, `std::cout << point` for printing. An overloaded operator is just a function with a special name, `operator+`, `operator[]` and so on.

10.1 The rules

- At least one operand must be a user-defined type. You cannot change what `1 + 2` means.
- You cannot invent new operators, or change an operator's precedence, associativity or number of operands.
- These cannot be overloaded: `::` `.` `.*` `?:` `sizeof` `typeid`.
- `=`, `[]`, `()` and `->` must be member functions.

Key idea

Overload an operator only when its meaning is obvious. `+` on a `Money` type should add amounts. If a reader would have to look up what your operator does, use a named function instead.

10.2 Member or non-member?

For `a @ b` (where `@` is any binary operator), the compiler tries both `a.operator@(b)` (member) and `operator@(a, b)` (non-member).

- A **member** operator always has the class object on its *left*. It is natural for operators that modify the object: `+=`, `-=`, `++`, `=`, `[]`.
- A **non-member** operator treats both sides symmetrically. Use it for `+`, `-`, `==`, `<`, and whenever the left operand is not your class, as with `std::cout << x`, where the left side is an `std::ostream`.

To see why symmetry matters: if `Money` has a non-explicit constructor from `double` and `operator+` is a member, then `m + 5.0` works but `5.0 + m` does not, because `5.0` has no member functions. A non-member `operator+` accepts both.

10.3 A complete example

The common pattern: implement the compound operator (`+=`) as a member, then write the plain operator (`+`) as a non-member in terms of it.

```
#include <iostream>
#include <compare>

class Complex {
```

```
public:
    constexpr Complex(double re = 0.0, double im = 0.0) : re_{re}, im_{im} {}

    double re() const { return re_; }
    double im() const { return im_; }

    // compound assignment: member, modifies *this, returns *this by reference
    Complex& operator+=(const Complex& rhs) {
        re_ += rhs.re_; im_ += rhs.im_;
        return *this;
    }
    Complex& operator*=(const Complex& rhs) {
        double r{re_ * rhs.re_ - im_ * rhs.im_};
        double i{re_ * rhs.im_ + im_ * rhs.re_};
        re_ = r; im_ = i;
        return *this;
    }

    // unary minus: returns a new value
    Complex operator-() const { return Complex{-re_, -im_}; }

    // C++20: compiler writes == and != for us, member by member
    bool operator==(const Complex&) const = default;

private:
    double re_, im_;
};

// binary operators: non-members, take lhs by value (we need a copy anyway)
Complex operator+(Complex lhs, const Complex& rhs) { return lhs += rhs; }
Complex operator*(Complex lhs, const Complex& rhs) { return lhs *= rhs; }

// stream output: must be non-member (left side is std::ostream)
std::ostream& operator<<(std::ostream& os, const Complex& c) {
    return os << '(' << c.re() << ", " << c.im() << "i)";
}

int main() {
    Complex a{1, 2}, b{3, -1};
    std::cout << a + b << '\n';       // (4, 1i)
    std::cout << a * b << '\n';       // (5, 5i)
    std::cout << 2.0 + a << '\n';     // (3, 2i): works because + is a non-member
    std::cout << (a == a) << '\n';    // 1
}
```

Notice that `operator<<` returns the stream by reference. That is what makes chaining work: `std::cout << a << b` is `(std::cout << a) << b`.

Here `operator<<` only uses the public getters. When it needs private members, declare it a `friend` inside the class (Chapter 9).

10.4 Input: `operator>>`

```
std::istream& operator>>(std::istream& is, Complex& c) {
    double re{}, im{};
    if (is >> re >> im) c = Complex{re, im};   // only change c if reading succeeded
    return is;
```

```
}
```

10.5 Comparison operators and the spaceship (C++20)

Before C++20 you wrote six comparison operators by hand. Now you can write one: the three-way comparison `operator<=>`, nicknamed "the spaceship". It reports less, equal or greater, and the compiler derives `<`, `<=`, `>` and `>=` from it. Defaulting it compares members in declaration order.

```
struct Version {
    int major, minor, patch;
    auto operator<=>(const Version&) const = default;   // also gives == and !=
};

Version{1, 4, 2} < Version{1, 10, 0};   // true: compares major, then minor, then
    patch
```

10.6 Increment and decrement

Prefix and postfix forms are told apart by a dummy `int` parameter in the postfix version.

```
class Counter {
public:
    Counter& operator++() {          // prefix: ++c
        ++value_;
        return *this;                // return the updated object
    }
    Counter operator++(int) {        // postfix: c++
        Counter old{*this};          // save a copy
        ++value_;
        return old;                  // return the OLD value
    }
private:
    int value_{0};
};
```

Postfix has to make a copy, so it is slower for complex types. Prefer `++i` unless you need the old value.

10.7 Subscript and function-call operators

```
class Matrix {
public:
    Matrix(std::size_t r, std::size_t c) : rows_{r}, cols_{c}, data_(r * c) {}

    double& operator()(std::size_t i, std::size_t j)       { return data_[i * cols_ +
    j]; }
    double  operator()(std::size_t i, std::size_t j) const { return data_[i * cols_ +
    j]; }
private:
    std::size_t rows_, cols_;
    std::vector<double> data_;
};
```

```
Matrix m{3, 3};
m(1, 2) = 5.0;       // returns a reference, so we can assign through it
```

`operator[]` takes exactly one argument before C++23, which is why matrix classes often use `operator()` for two indices. A class with `operator()` is called a *function object* or *functor*; Chapter 17 covers them.

10.8 Conversion operators

```
class Connection {
public:
    explicit operator bool() const { return open_; }   // lets you write if (conn)
private:
    bool open_{false};
};

Connection conn;
if (conn) { /* ... */ }          // OK: explicit bool conversion allowed in conditions
// int n = conn;                  // error, thanks to explicit
```

Mark conversion operators `explicit`, for the same reason as single-argument constructors: implicit conversions cause surprising overload choices.

10.9 Assignment

`operator=` was covered with copying in Chapter 8. Return `*this` by reference, handle self-assignment, and prefer to let the compiler generate it (the Rule of Zero).

Chapter 11

Namespaces and Program Organisation

11.1 The problem namespaces solve

Large programs combine code from many people and libraries. If two libraries both define a function called `log` or a class called `Matrix`, the linker cannot tell them apart. A *namespace* is a named region that groups declarations, so their full names no longer clash.

```
namespace physics {
    constexpr double g{9.81};
    double fallTime(double height) { return std::sqrt(2 * height / g); }
}

namespace finance {
    double fallTime(double price);   // a different function, no clash
}

double t{physics::fallTime(20.0)};   // :: is the scope resolution operator
```

Namespaces can be reopened (a namespace may be spread over many files) and nested:

```
namespace company::pricing {         // C++17 shorthand for nested namespaces
    double blackScholes(double s, double k);
}
namespace cp = company::pricing;     // a namespace alias
cp::blackScholes(100, 95);
```

A name with no namespace at all lives in the *global namespace*; you can name it explicitly with a leading `::`, as in `::fallTime`.

11.2 `using` declarations and `using` directives

```
using std::cout;            // using-DECLARATION: brings in one name
cout << "hi\n";

using namespace std;        // using-DIRECTIVE: brings in every name in std
```

Never write using namespace in a header

A `using namespace std;` in a header leaks every standard-library name into every file that includes it, where those names can silently collide with the user's own (`count`, `distance`, `max`, `data` are all in `std`). Inside a `.cpp` file, or better inside a single function, a using-declaration for a few names is fine.

Best practice

Write `std::` explicitly. It costs five characters and makes it obvious where every name comes from.

11.3 Argument-dependent lookup

Why does `std::cout << x` find an `operator<<` defined in your namespace, even though you never qualified it? Because of *argument-dependent lookup* (ADL): when you call an unqualified function, the compiler also searches the namespaces of the arguments' types. So put a type's operators and helper functions in the same namespace as the type, and they will be found automatically.

11.4 Linkage: what is visible across files

- **External linkage**: the name can be used from other `.cpp` files (if they declare it). Normal functions and non-const global variables have external linkage.
- **Internal linkage**: the name is private to its own `.cpp` file. Use this for helpers that no other file needs.

The modern way to get internal linkage is an *unnamed namespace*:

```
// parser.cpp
namespace {                          // everything here is private to parser.cpp
    bool isDelimiter(char c) { return c == ',' || c == ';'; }
}
std::vector<std::string> split(const std::string& s) { /* uses isDelimiter */ }
```

Marking a function `static` at file scope does the same thing (the older C way).

11.5 Global variables

Non-const global variables

A global variable that anything can modify makes a program hard to reason about: any function anywhere might have changed it, so you cannot understand a function by reading it alone. Global *constants* are fine (use `inline constexpr` in a header, Chapter 3). For mutable state, pass it explicitly or keep it inside an object.

11.6 A typical project layout

```
project/
  CMakeLists.txt        # build description
  include/project/      # public headers (.h / .hpp): declarations, classes, templates
  src/                  # implementation files (.cpp): definitions
  tests/                # unit tests
```

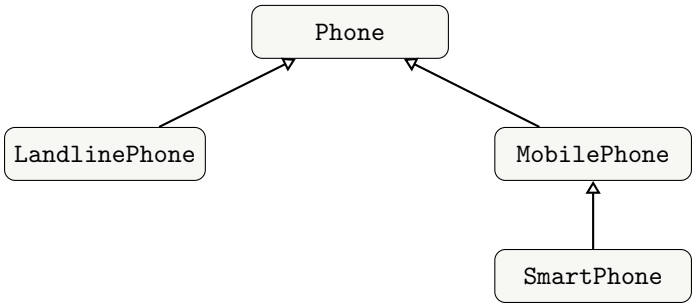
Each header should include everything it needs to compile on its own, and nothing more. Prefer to include headers in a `.cpp` file rather than in another header when you can, to keep compile times down.

Chapter 12

Inheritance

12.1 The idea: “is-a”

Inheritance creates a new class (the *derived* class) from an existing one (the *base* class). The derived class gets all the base class’s members and can add its own. Use it when the relationship is genuinely “is-a”: a smartphone *is a* phone; a savings account *is an* account.




```
class Phone {
public:
    explicit Phone(std::string number) : number_{std::move(number)} {}
    void call(const std::string& to) const {
        std::cout << number_ << " calling " << to << '\n';
    }
    const std::string& number() const { return number_; }
private:
    std::string number_;
};

class MobilePhone : public Phone {                // MobilePhone inherits from Phone
public:
    MobilePhone(std::string number, int battery)
        : Phone{std::move(number)}, battery_{battery} {}   // construct the base part
    first
    void sendSms(const std::string& to, const std::string& text) const {
        std::cout << number() << " -> " << to << ": " << text << '\n';
    }
private:
    int battery_;
};

MobilePhone m{"98300-12345", 80};
m.call("Mom");              // inherited from Phone
m.sendSms("Mom", "Hi");     // added by MobilePhone
```

A `MobilePhone` object contains a complete `Phone` object inside it (the *base subobject*), plus its own members. Note that `sendSms` cannot read `number_` directly, because it is `private` to `Phone`; it uses the public `number()` instead.

12.2 Construction and destruction order

Key idea

Construction goes from the **base down**: base class first, then the derived class's members, then the derived constructor's body. Destruction runs in exactly the **reverse** order. A derived class may rely on its base being fully built, and the base must outlive everything that depends on it.

If you do not name a base constructor in the initialiser list, the base's *default* constructor is used, and compilation fails if it has none. That is why `MobilePhone` explicitly writes `: Phone{...}`.

12.3 `protected` members

`protected` sits between `public` and `private`: derived classes can access it, outside code cannot.

```
class Account {
protected:
    double balance_{0.0};           // derived classes may use this directly
};
class SavingsAccount : public Account {
public:
    void addInterest(double rate) { balance_ *= (1 + rate); }   // OK
};
```

Use `protected` data sparingly. Every derived class written by anyone can then break the base class's invariants. Protected *member functions* for derived classes to call are usually the better design.

12.4 Public, protected and private inheritance

The keyword before the base class name limits how inherited members appear to the outside world:

Member in base	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

Only **public** inheritance models "is-a", and it is what you will use 99% of the time. Private inheritance means "is implemented in terms of", and composition (a data member) usually expresses that more clearly. If you omit the keyword, a `class` inherits privately and a `struct` publicly, so always write it.

12.5 Overriding and hiding

A derived class can define a function with the same name as a base-class function:

```
class Base {
public:
    void greet() const { std::cout << "Base\n"; }
```

```
    void print(int x) const;
};
class Derived : public Base {
public:
    void greet() const {
        Base::greet();                // call the base version explicitly
        std::cout << "Derived\n";
    }
    void print(const std::string& s) const;
};

Derived d;
d.greet();        // Base, then Derived
// d.print(5);    // ERROR: Derived::print hides ALL Base::print overloads
```

Name hiding

A function in a derived class *hides* every base-class function with the same name, even ones with different parameters. To bring the base overloads back, add `using Base::print;` inside `Derived`.

Without the `virtual` keyword, which version gets called depends on the *declared* type of the variable, not on the actual object. That is often not what you want, and it is exactly the problem the next chapter solves.

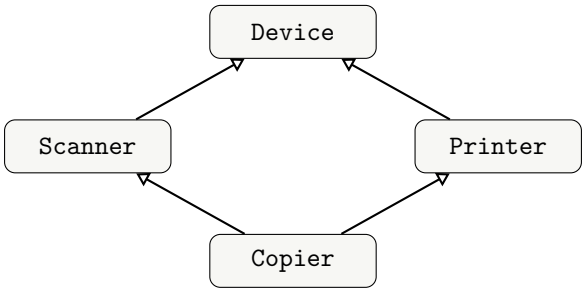
12.6 Multiple inheritance

A class may have several base classes:

```
class Camera { public: void takePhoto(); };
class MusicPlayer { public: void play(); };
class SmartPhone : public MobilePhone, public Camera, public MusicPlayer { };
```

If two bases have a member with the same name, calls are ambiguous and must be qualified (`sp.Camera::reset()`).

12.6.1 The diamond problem and virtual base classes



If `Scanner` and `Printer` both inherit from `Device`, a `Copier` normally contains *two* `Device` subobjects, and `copier.powerOn()` is ambiguous. Virtual inheritance tells the compiler to keep just one shared copy:

```
class Device { public: void powerOn(); };
class Scanner : virtual public Device { };
```

```
class Printer : virtual public Device { };
class Copier  : public Scanner, public Printer { };   // one Device inside

Copier c;
c.powerOn();    // unambiguous
```

With virtual inheritance, the *most derived* class (`Copier`) is responsible for constructing the shared `Device`.

Best practice

Multiple inheritance of *interfaces* (abstract classes with only pure virtual functions, Chapter 13) is common and safe. Multiple inheritance of classes with data is rarely worth the complexity.

12.7 Composition versus inheritance

Before inheriting, ask whether the relationship is "is-a" or "has-a". A car *has an* engine; it is not an engine. Express "has-a" with a data member (composition):

```
class Car {
    Engine engine_;      // composition: Car has an Engine
    std::vector<Wheel> wheels_;
};
```

Key idea

Prefer composition over inheritance. It couples classes more loosely and is easier to change. Use public inheritance when you need polymorphism: when code should work with any kind of `Phone` through a common interface.

Chapter 13

Polymorphism and Virtual Functions

Polymorphism ("many forms") means writing code against a common interface and letting each concrete type supply its own behaviour. It is the main reason to use inheritance at all.

13.1 Base-class pointers and references

Because a `Circle` *is a* `Shape`, a `Shape*` or `Shape&` may refer to a `Circle`. Converting derived to base like this is called *upcasting*, and it is always safe and implicit.

```
class Shape {
public:
    double area() const { return 0.0; }          // NOT virtual (yet)
};
class Circle : public Shape {
public:
    explicit Circle(double r) : r_{r} {}
    double area() const { return 3.14159 * r_ * r_; }
private:
    double r_;
};

Circle c{2.0};
Shape& s{c};                 // upcast: fine
std::cout << s.area();       // prints 0, not 12.57!
```

The call printed 0 because, without `virtual`, the compiler picks the function from the *static type* of the expression (`Shape`), decided at compile time. This is **static binding** (also called early binding).

13.2 Virtual functions: dynamic binding

Declare the function `virtual` in the base class, and the call is resolved at *runtime* from the *dynamic type*: the actual type of the object. This is **dynamic binding** (late binding).

```
class Shape {
public:
    virtual double area() const { return 0.0; }
    virtual ~Shape() = default;                        // see "virtual destructors"
    below
};
class Circle : public Shape {
public:
    explicit Circle(double r) : r_{r} {}
    double area() const override { return 3.14159 * r_ * r_; }
private:
    double r_;
};
class Square : public Shape {
public:
    explicit Square(double s) : s_{s} {}
```

```
    double area() const override { return s_ * s_; }
private:
    double s_;
};

void report(const Shape& s) { std::cout << s.area() << '\n'; }

report(Circle{2.0});    // 12.5664
report(Square{3.0});    // 9
```

`report` was written once, against `Shape`, and works for every shape, including shapes invented years later. That is the power of polymorphism: new behaviour without changing existing code.

Key idea

Polymorphism needs three things together: **public inheritance**, a **virtual** function, and a call through a **pointer or reference** to the base. Calling on an object directly (`circle.area()`) uses its own type anyway.

13.2.1 `override` and `final`

`override` tells the compiler "this is meant to override a base virtual function". If it does not (because of a typo, a different parameter type, or a missing `const`), you get a compile error instead of a silent new, unrelated function.

```
class Square : public Shape {
    double area() override;          // ERROR: base is "area() const"; without
    override
};                                    // this would silently NOT override
```

`final` prevents further overriding of a function, or further derivation from a class: `class Square final : public Shape { ... };`.

Best practice

Write `virtual` only in the base class, and `override` on every overriding function in derived classes.

13.3 Virtual destructors

```
Shape* p{new Circle{1.0}};
delete p;      // which destructor runs?
```

If `Shape`'s destructor is not virtual, `delete p` calls only `~Shape()`. The `Circle` part is never destroyed, and formally the behaviour is undefined. With a virtual destructor, `~Circle()` runs first and then `~Shape()`, as it should.

A polymorphic base class needs a virtual destructor

Rule: if a class has any virtual function, give it a public virtual destructor (`virtual ~Base() = default;`). If a class is not meant to be deleted through a base pointer, make the base destructor protected and non-virtual instead.

13.4 Pure virtual functions and abstract classes

Often the base class has no sensible implementation: what is the area of a generic "shape"? Make the function *pure virtual* with `= 0`.

```
class Shape {
public:
    virtual double area() const = 0;        // pure virtual: no body needed
    virtual std::string name() const = 0;
    virtual ~Shape() = default;
};

// Shape s;      // ERROR: cannot create an object of an abstract class
```

A class with at least one pure virtual function is **abstract**: you cannot create objects of it, only of derived classes that override every pure virtual function. A class with *only* pure virtual functions and no data is an **interface**: a pure contract that says what a type can do, not how.

```
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(1.0));
shapes.push_back(std::make_unique<Square>(2.0));

double total{0.0};
for (const auto& s : shapes) total += s->area();   // each calls its own area()
```

This is the standard way to hold a collection of different types: a container of (smart) pointers to the base.

13.5 Object slicing

```
Circle c{2.0};
Shape s = c;    // only if Shape were concrete: COPIES just the Shape part
```

Copying a derived object into a base-class *object* (not a pointer or reference) keeps only the base part; the derived data and behaviour are "sliced off". The same happens when you pass by value (`void f(Shape s)`) or store `std::vector<Shape>`. Always handle polymorphic objects through references or pointers.

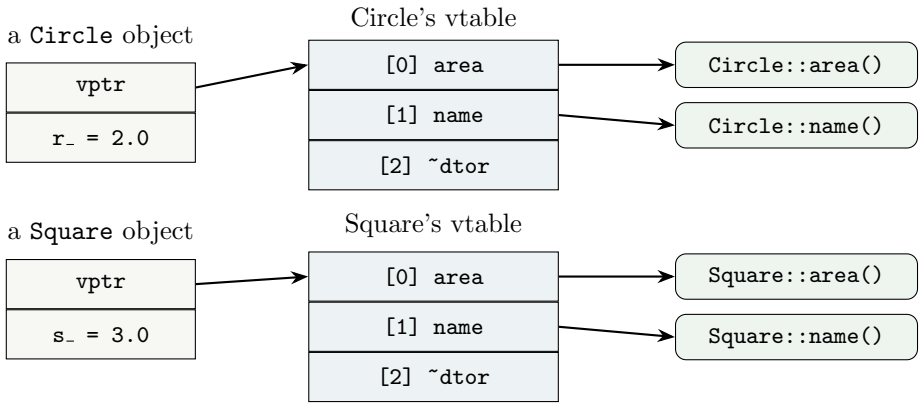
13.6 How it works: the virtual table

Dynamic binding is not magic, and understanding the mechanism explains its costs and rules. The details are compiler-specific, but every major compiler works essentially like this.

13.6.1 The vtable and the vptr

1. For each class that has virtual functions, the compiler builds one static table, the **virtual table** (*vtable*). It is an array of function pointers, one slot per virtual function, holding the address of the version that class uses.
2. Each *object* of such a class carries one hidden pointer, the **vptr**, set by the constructor to point to its class's vtable.

3. A virtual call `s.area()` compiles to: follow `s`'s vptr to its vtable, load the function pointer from `area`'s slot, and call it.



Every `Circle` shares the same vtable; only the vptr is per object. A derived class's vtable starts as a copy of its base's layout: slots for functions it overrides point to its own versions, slots it does not override keep pointing to the base versions, and any new virtual functions get extra slots at the end. Because `area` is in slot 0 for every class in the hierarchy, code compiled only against `Shape` can call slot 0 on any object and reach the right function.

13.6.2 What this explains

- **Size**: an object of a class with virtual functions is one pointer (8 bytes on 64-bit systems) larger.
- **Speed**: a virtual call adds two memory loads and an indirect jump, and usually prevents inlining. For most code this is negligible; in a tight inner loop running billions of times, it can matter.
- **Calling virtual functions in constructors**: while `Shape`'s constructor runs, the vptr still points to `Shape`'s vtable, because the `Circle` part does not exist yet. So a virtual call made inside a base constructor or destructor uses the *base* version. Avoid virtual calls there.
- **Why pointers or references are required**: a sliced `Shape` object has a `Shape` vptr, so there is no `Circle` behaviour left to find.

13.7 Runtime versus compile-time polymorphism

Virtual functions give *runtime* polymorphism: the type is chosen while the program runs. Templates (Chapter 16) give *compile-time* polymorphism: the code is generated for each type, with no vtable and full inlining, but every type must be known while compiling. Performance-critical libraries often prefer templates; plugin systems and heterogeneous collections need virtual functions.

Chapter 14

Type Conversion and Casting

14.1 Implicit conversions

The compiler converts between types automatically in many situations:

```
int i{5};
double d = i;          // int -> double: safe, no information lost
long long big = i;     // widening: safe
int j = 3.99;          // double -> int: truncates to 3 (warning with -Wall)
unsigned u = -1;       // wraps to 4294967295: almost always a bug
```

In arithmetic, smaller types are first *promoted* (`char` and `short` to `int`), and mixed operands are converted to a common type (`int + double` becomes `double`). Conversions that may lose information (`double` to `int`, large to small integer types) are *narrowing*; brace initialisation forbids them (Chapter 2).

14.2 Why C-style casts are dangerous

In C, `(int)x` or `(Derived*)p` performs *whatever conversion it takes* to make the code compile: a numeric conversion, removing `const`, reinterpreting raw bits, or a mixture. A single typo can turn a safe conversion into a dangerous one with no warning, and C-style casts are almost impossible to search for in a large codebase.

C++ splits casting into four named operators. Each does one kind of conversion, refuses the others, and is easy to find with a text search.

Cast	What it does	Checked?
`static_cast<T>(x)`	Ordinary, well-defined conversions: numeric types, upcasts, unchecked downcasts, from `void*`	At compile time
`dynamic_cast<T>(x)`	Safe downcast in a polymorphic hierarchy	At runtime
`const_cast<T>(x)`	Adds or removes `const` (or `volatile`)	No
`reinterpret_cast<T>(x)`	Treats the bits as a different type: pointer to integer, unrelated pointers	No

14.3 `static_cast`

The everyday cast. Use it to make an intended conversion explicit:

```
int total{7}, count{2};
double avg{static_cast<double>(total) / count};   // 3.5, not 3

enum class Side { buy, sell };
int code{static_cast<int>(Side::sell)};           // 1

char c{'A'};
std::cout << static_cast<int>(c);                 // 65
```

In a class hierarchy, `static_cast` can also *downcast* (base to derived), but it does **not** check that the object really is of the derived type. If you are wrong, the behaviour is undefined.

```
Shape* s{new Circle{1.0}};
Circle* c{static_cast<Circle*>(s)};    // fine ONLY because s really points to a
    Circle
```

14.4 dynamic_cast

`dynamic_cast` performs a *checked* downcast using runtime type information (RTTI). It works only on polymorphic classes (classes with at least one virtual function), because it reads the type information reached through the object's vptr.

```
void process(Shape* s) {
    if (auto* c = dynamic_cast<Circle*>(s)) {      // pointer form: nullptr on failure
        std::cout << "circle with radius " << c->radius() << '\n';
    } else {
        std::cout << "not a circle\n";
    }
}

void processRef(Shape& s) {
    try {
        Circle& c = dynamic_cast<Circle&>(s);      // reference form: throws on
    failure
    } catch (const std::bad_cast&) {
        std::cout << "not a circle\n";
    }
}
```

Best practice

Frequent `dynamic_cast` is a design smell: it means code is asking "what type are you?" instead of letting virtual functions answer. Prefer adding a virtual function to the base class. Keep `dynamic_cast` for the genuine exceptions, such as checking whether an object also supports an optional interface.

`typeid(x)` (from `<typeinfo>`) also uses RTTI and returns information about an object's dynamic type; `typeid(x) == typeid(Circle)` tests for an exact type match.

14.5 const_cast

`const_cast` removes (or adds) `const`. Its one legitimate everyday use is calling an old C-style API that takes a non-const pointer but does not actually modify the data:

```
void legacyPrint(char* s);                    // old API: does not really modify s
const std::string msg{"hello"};
legacyPrint(const_cast<char*>(msg.c_str())); // OK as long as legacyPrint never writes
```

Modifying a truly const object is undefined behaviour

If an object was *defined* as `const`, writing to it through a pointer obtained with `const_cast` is undefined behaviour. The compiler may have put it in read-only memory or assumed it never

changes.

14.6 `reinterpret_cast`

`reinterpret_cast` tells the compiler to treat a value's bits as another type. It is for low-level work: talking to hardware, converting a pointer to an integer for hashing, or handling raw byte buffers.

```
int x{42};
auto address{reinterpret_cast<std::uintptr_t>(&x)};   // pointer -> integer
```

Reading an object through a pointer to an unrelated type usually breaks the *strict aliasing* rule and is undefined behaviour. To reinterpret the bits of one value as another type (say, a `double`'s bits as a `uint64_t`), use `std::bit_cast` (C++20) or `std::memcpy`, which are well defined.

Key idea

Choose the weakest cast that works: `static_cast` for normal conversions, `dynamic_cast` when a downcast might fail, `const_cast` only for old APIs, `reinterpret_cast` only for genuinely low-level code. Never use C-style casts in C++.

Chapter 15

Exceptions

15.1 Why exceptions?

In C, functions report errors through return codes. That has three weaknesses: callers can forget to check; every function between the error and the code that can handle it must pass the code along by hand; and constructors have no return value at all, so they cannot report failure that way.

Exceptions separate *detecting* an error from *handling* it. The function that finds a problem `throw`s; control jumps directly to the nearest matching `catch`, however many calls up the stack it is.

15.2 `throw`, `try` and `catch`

```
#include <stdexcept>

double safeDivide(double a, double b) {
    if (b == 0.0) throw std::invalid_argument("division by zero");
    return a / b;
}

int main() {
    try {
        std::cout << safeDivide(10, 2) << '\n';   // 5
        std::cout << safeDivide(1, 0) << '\n';    // throws; the next line never runs
        std::cout << "not printed\n";
    } catch (const std::invalid_argument& e) {
        std::cout << "Error: " << e.what() << '\n';
    }
    std::cout << "program continues\n";
}
```

- `throw expr;` creates an exception object and starts looking for a handler.
- A `try` block marks code whose exceptions you want to handle.
- `catch` clauses are tried *in order*; the first whose type matches wins.
- If no handler anywhere matches, `std::terminate` is called and the program ends.

Best practice

Throw by value, catch by `const` reference: `catch (const std::exception& e)`. Catching by value copies the exception and slices derived exception types down to the base.

15.3 Stack unwinding: what happens on the way up

This is the part of exceptions that matters most, so it is worth slowing down. Suppose `main` calls `a`, which calls `b`, which throws:

```
struct Tracer {
    std::string name;
    ~Tracer() { std::cout << "destroying " << name << '\n'; }
};

void b() {
    Tracer tb{"tb"};
    throw std::runtime_error("failure in b");
}
void a() {
    Tracer ta{"ta"};
    b();
    std::cout << "never printed\n";
}
int main() {
    try { a(); }
    catch (const std::exception& e) { std::cout << "caught: " << e.what() << '\n'; }
}
```

Output:

```
destroying tb
destroying ta
caught: failure in b
```

When an exception leaves a function, that function's stack frame is removed and *the destructors of all its fully constructed local objects run*, in reverse order of construction. This repeats function by function until a matching `catch` is found. The process is called **stack unwinding**.

Exceptions and RAII belong together

Because destructors run during unwinding, any resource owned by an object (memory in a `std::vector`, a file in an `std::ifstream`, a lock in a `std::lock_guard`) is released automatically when an exception passes through. A raw `new` without a matching owner object leaks. This is the strongest reason to follow RAII.

Destructors must not throw

If a destructor throws while the stack is already unwinding from another exception, the program calls `std::terminate`. Destructors are implicitly `noexcept`; never let an exception escape one.

15.4 The standard exception hierarchy

All standard exceptions derive from `std::exception`, which provides the virtual `what()` message.

Exception	Typical cause
`std::logic_error`	A bug that could have been prevented by checking first
`  std::invalid_argument`	An argument with an invalid value
`  std::out_of_range`	An index out of range, e.g. `vector::at`
`std::runtime_error`	Errors detectable only while running
`  std::overflow_error`, `std::range_error`	Arithmetic problems
`std::bad_alloc`	`new` could not get memory
`std::bad_cast`	A failed `dynamic_cast` to a reference

Because catching works with inheritance, `catch (const std::exception& e)` catches all of them. Order your `catch` clauses from most specific to most general; a general handler placed first would catch everything.

```
try { /* ... */ }
catch (const std::out_of_range& e)  { /* specific */ }
catch (const std::exception& e)     { /* any standard exception */ }
catch (...)                         { /* absolutely anything else */ }
```

15.5 Your own exception types

Derive from a standard exception so that general handlers still catch yours:

```
class InsufficientFunds : public std::runtime_error {
public:
    InsufficientFunds(double needed, double available)
        : std::runtime_error{"insufficient funds"},
          needed_{needed}, available_{available} {}
    double shortfall() const { return needed_ - available_; }
private:
    double needed_, available_;
};

try {
    account.withdraw(1000);
} catch (const InsufficientFunds& e) {
    std::cout << e.what() << ", short by " << e.shortfall() << '\n';
}
```

15.6 Rethrowing

A handler can do partial work (such as logging) and pass the exception on with a bare `throw;`, which rethrows the *original* exception object unchanged:

```
catch (const std::exception& e) {
    log(e.what());
    throw;          // NOT "throw e;", which would copy and slice it
}
```

15.7 `noexcept`

`noexcept` on a function promises it will not throw. If it throws anyway, `std::terminate` is called.

```
void swap(Buffer& other) noexcept;
```

It matters for performance as well as documentation: `std::vector` moves its elements when it grows only if their move constructor is `noexcept`; otherwise it falls back to slower copies to keep its own guarantees (Chapter 18). Mark move constructors, move assignment, `swap` and destructors `noexcept`.

15.8 Exception-safety guarantees

When a function throws, what state is left behind? Libraries describe this with three levels:

- **No-throw guarantee**: the operation never throws (`noexcept`).
- **Strong guarantee**: if it throws, the program state is exactly as before, as if nothing happened. The copy-and-swap idiom (Chapter 8) gives this.
- **Basic guarantee**: if it throws, nothing leaks and all objects remain valid, but their values may have changed.

Every function should at least give the basic guarantee. RAII gets you most of the way there automatically.

15.9 When not to use exceptions

Exceptions are for *exceptional* situations: a missing file, a lost connection, invalid input deep inside a calculation. For outcomes that are normal and expected ("key not found in the map"), return a value instead: `std::optional`, a `bool`, or C++23's `std::expected`, which holds either a result or an error. Some low-latency and embedded codebases disable exceptions altogether because of their unpredictable timing on the error path.

Chapter 16

Templates

16.1 The problem templates solve

Suppose you need `max` for `int`, `double` and `std::string`. With overloading you write the same body three times, and again for every new type. In C you might use macros or `void*`, losing type safety. Templates let you write the code once, with the type as a parameter, and the compiler produces a correctly typed version for each type you use.

Key idea

A template is not code; it is a **recipe for code**. Nothing is generated until you use the template with specific types. Then the compiler *instantiates* it: it substitutes the types and compiles the result as if you had written that version by hand.

16.2 Function templates

```
template <typename T>
T maxOf(T a, T b) {
    return (a > b) ? a : b;
}

maxOf(3, 7);                                    // T = int    -> instantiates
    maxOf<int>
maxOf(2.5, 1.5);                                // T = double -> instantiates
    maxOf<double>
maxOf(std::string{"apple"}, std::string{"pear"});  // T = std::string
maxOf<double>(3, 2.5);                          // explicit: T = double, 3 converts to
    3.0
// maxOf(3, 2.5);                               // ERROR: T cannot be both int and
    double
```

`typename T` declares a *template type parameter*; `class T` means exactly the same thing. The compiler usually works out `T` from the arguments (*template argument deduction*). Deduction does not perform conversions, which is why `maxOf(3, 2.5)` fails: one argument says `int`, the other `double`.

The template places requirements on `T` implicitly: `maxOf` needs `T` to support `>` and copying. Use it with a type lacking `>` and you get a compile error from inside the template, often a long and confusing one. Concepts (Section 16.9) fix that.

For larger types, take parameters by const reference:

```
template <typename T>
const T& maxOf(const T& a, const T& b) { return (a > b) ? a : b; }
```

16.2.1 Several type parameters

```
template <typename A, typename B>
auto add(A a, B b) {            // return type deduced from the expression
    return a + b;
}
add(3, 2.5);                    // A = int, B = double, returns double 5.5
```

16.2.2 Abbreviated function templates (C++20)

Writing `auto` as a parameter type makes a function template automatically:

```
auto add(auto a, auto b) { return a + b; }   // same as the version above
```

16.3 Non-type template parameters

A template parameter can also be a compile-time *value*, such as an integer:

```
template <typename T, std::size_t N>
constexpr std::size_t arraySize(const T (&)[N]) { return N; }

int a[17]{};
static_assert(arraySize(a) == 17);

template <int Power>
double raise(double x) {
    double r{1.0};
    for (int i{0}; i < Power; ++i) r *= x;
    return r;
}
raise<3>(2.0);    // 8.0; Power is fixed at compile time
```

`std::array<int, 5>` is the best-known example: the size `5` is a non-type template parameter.

16.4 Class templates

Containers are the classic use. Here is a simple fixed-capacity stack:

```
template <typename T, std::size_t Capacity = 100>
class Stack {
public:
    void push(const T& value) {
        if (size_ == Capacity) throw std::overflow_error("stack full");
        data_[size_++] = value;
    }
    T pop();                                    // defined below
    bool empty() const { return size_ == 0; }
    std::size_t size() const { return size_; }
private:
    std::array<T, Capacity> data_{};
    std::size_t size_{0};
};

// Defining a member outside the class repeats the template header:
template <typename T, std::size_t Capacity>
```

```
T Stack<T, Capacity>::pop() {
    if (empty()) throw std::underflow_error("stack empty");
    return data_[--size_];
}

Stack<int> s;              // T = int, Capacity = 100 (default)
s.push(4);
Stack<std::string, 10> names;
```

Each distinct set of arguments is a completely separate type: `Stack<int>` and `Stack<double>` share no code and cannot be assigned to each other.

16.4.1 Class template argument deduction (CTAD)

Since C++17, the compiler can deduce class template arguments from a constructor call:

```
std::pair p{1, 2.5};          // std::pair<int, double>
std::vector v{1, 2, 3};       // std::vector<int>
```

16.5 Why templates live in header files

This rule surprises everyone once. To instantiate `maxOf<double>`, the compiler needs the template's full *definition*, not just a declaration, in the same translation unit as the call. If you declare a template in a header and define it in a `.cpp` file, other files see only the declaration, nothing gets instantiated, and you get "undefined reference" linker errors.

Best practice

Put the whole template, including member function bodies, in the header. This does not violate the One Definition Rule: templates are exempt, like `inline` functions.

16.6 Template specialisation

Sometimes one type needs different behaviour. A *full specialisation* provides a custom version for exactly one set of arguments:

```
template <typename T>
struct TypeName { static constexpr const char* value = "unknown"; };

template <>                                  // full specialisation for int
struct TypeName<int> { static constexpr const char* value = "int"; };

template <>
struct TypeName<double> { static constexpr const char* value = "double"; };

std::cout << TypeName<int>::value;           // int
std::cout << TypeName<char>::value;          // unknown
```

A *partial specialisation* (class templates only) handles a whole family of arguments, such as "any pointer type":

```
template <typename T>
struct IsPointer { static constexpr bool value = false; };
```

```
template <typename T>
struct IsPointer<T*> { static constexpr bool value = true; };   // matches every T*

static_assert(IsPointer<int*>::value);
static_assert(!IsPointer<int>::value);
```

This pattern, computing facts about types at compile time, is how the standard *type traits* in `<type_traits>` work: `std::is_pointer_v<T>`, `std::is_integral_v<T>`, `std::is_same_v<A, B>`, and many more.

Specialise function templates rarely

Full specialisation of *function* templates has surprising interactions with overload resolution. To give a function different behaviour for one type, write an ordinary overload or use `if constexpr` instead.

16.7 Alias templates

```
template <typename T>
using Matrix = std::vector<std::vector<T>>;

Matrix<double> m(3, std::vector<double>(3));
```

16.8 Variadic templates and fold expressions

A template can take any number of arguments using a *parameter pack* (`...`):

```
template <typename... Args>
auto sum(Args... args) {
    return (args + ...);        // fold expression (C++17): a1 + a2 + ... + an
}
sum(1, 2, 3, 4);                // 10

template <typename... Args>
void printAll(const Args&... args) {
    ((std::cout << args << ' '), ...);   // print each argument
    std::cout << '\n';
}
printAll("x =", 3, "y =", 4.5);
```

This is how `std::make_unique` and `emplace_back` forward any number of constructor arguments.

16.9 Concepts: constraining templates (C++20)

A *concept* is a named, compile-time requirement on a type. Constraining a template with one documents what it needs, and turns pages of confusing errors into one clear message.

```
#include <concepts>

template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;
```

```
template <Numeric T>
T average(const std::vector<T>& v) {
    T sum{};
    for (const T& x : v) sum += x;
    return v.empty() ? T{} : sum / static_cast<T>(v.size());
}

average(std::vector<double>{1.0, 2.0});        // OK
// average(std::vector<std::string>{"a"});      // clear error: std::string is not
    Numeric

// A concept defined by the operations a type must support:
template <typename T>
concept Printable = requires(std::ostream& os, const T& x) {
    { os << x };                                  // must be streamable
};

void show(const Printable auto& x) { std::cout << x << '\n'; }
```

The standard library provides many ready-made concepts in `<concepts>`: `std::integral`, `std::floating_point`, `std::same_as`, `std::copyable`, `std::totally_ordered`, and more.

Key idea

Templates give you generic code with no runtime cost: each instantiation is ordinary, fully optimised code for one type. The trade-offs are longer compile times, larger binaries if you instantiate many versions, and (without concepts) poor error messages.

Chapter 17

Functors, Lambdas and `std::function`

Much of the standard library takes *behaviour* as an argument: how to compare elements when sorting, which elements to count, what to do with each one. C++ has three ways to package behaviour: function pointers (Chapter 5), function objects, and lambdas.

17.1 Function objects (functors)

A *function object*, or *functor*, is an object of a class that overloads the function-call operator `operator()`. It can be called like a function, but because it is an object, it can also hold *state*.

```
class GreaterThan {
public:
    explicit GreaterThan(int threshold) : threshold_{threshold} {}
    bool operator()(int x) const { return x > threshold_; }
private:
    int threshold_;                 // state carried with the "function"
};

GreaterThan above10{10};
above10(15);                        // true: looks like a function call
above10(3);                         // false

std::vector<int> v{4, 12, 7, 30};
auto n = std::count_if(v.begin(), v.end(), GreaterThan{10});   // 2
```

A plain function cannot do this without a global variable: the threshold has to live somewhere. A functor carries it along.

Functors are also fast. When the compiler sees a call to `GreaterThan::operator()`, it knows exactly which function will run, so it can inline it. A call through a function pointer is often opaque to the optimiser.

The standard library provides ready-made functors in `<functional>`: `std::less`, `std::greater`, `std::plus`, `std::multiplies`, and others.

```
std::sort(v.begin(), v.end(), std::greater<>{});   // sort in descending order
std::set<int, std::greater<>> s;                   // a set ordered high to low
```

17.2 Lambdas

Writing a whole class for a one-line comparison is tedious. A *lambda expression* writes the functor for you, inline.

```
int threshold{10};
auto n = std::count_if(v.begin(), v.end(),
                       [threshold](int x) { return x > threshold; });
```

What a lambda really is

For each lambda, the compiler generates an unnamed class with an `operator()` containing the body, and the captured variables become its data members. The lambda expression creates an object of that class, called a *closure*. So the lambda above is exactly a `GreaterThan` functor that the compiler wrote for you.

17.2.1 Anatomy

```
[captures](parameters) -> ReturnType { body }
```

The `-> ReturnType` part is optional; the compiler deduces it from the `return` statements. If there are no parameters, `()` can be omitted too: `[]{ std::cout << "hi"; }`.

17.2.2 Captures

The capture list says which variables from the surrounding scope the lambda may use, and how.

Capture	Meaning
`[]`	Capture nothing
`[x]`	Capture `x` by value: the lambda stores its own copy, made when the lambda is created
`[&x]`	Capture `x` by reference: the lambda uses the original variable
`[=]`	Capture everything used, by value
`[&]`	Capture everything used, by reference
`[=, &x]`	Everything by value except `x` by reference
`[this]`	Capture the current object (to use members inside a member function)
`[y = expr]`	Init-capture: create a new member `y` initialised from an expression (C++14)
`[p = std::move(ptr)]`	Move a variable into the lambda, e.g. a `unique_ptr`

```
int counter{0};
auto byValue = [counter]() { return counter + 1; };   // uses the copy made now
auto byRef   = [&counter]() { ++counter; };           // modifies the original
byRef(); byRef();
std::cout << counter;        // 2
std::cout << byValue();      // 1: its copy was taken when counter was 0
```

A lambda's `operator()` is `const` by default, so it cannot modify its by-value copies. Add `mutable` to allow it: `[count]() mutable { return ++count; }`.

Dangling reference captures

A lambda that captures by reference must not outlive the variables it refers to. Returning such a lambda from a function, or storing it to run later (in another thread, a callback, an event queue), is a classic source of crashes. Capture by value when the lambda may outlive the current scope.

17.2.3 Generic lambdas

Using `auto` for a parameter makes the lambda's `operator()` a template:

```
auto print = [](const auto& x) { std::cout << x << '\n'; };
print(42);
print("hello");
print(3.14);
```

17.3 `std::function`: storing any callable

Every lambda has its own unique, unnamed type, so how do you store "some callable taking an `int` and returning `bool`" in a variable or a container? `std::function` can hold *any* callable with a matching signature: a function pointer, a functor, or a lambda.

```
#include <functional>

double half(double x) { return x / 2; }

std::function<double(double)> op;
op = [](double x) { return x * x; };        // a lambda
std::cout << op(3.0);                       // 9
op = half;                                  // a plain function
std::cout << op(9.0);                       // 4.5

std::vector<std::function<void()>> tasks;   // a list of different callables
tasks.push_back([] { std::cout << "task 1\n"; });
tasks.push_back([] { std::cout << "task 2\n"; });
for (auto& t : tasks) t();
```

`std::function` achieves this through *type erasure*: internally it uses an indirect call, much like a virtual function, and may allocate memory for large callables. That flexibility has a small cost.

Best practice

To *accept* a callable as a function parameter, use a template parameter (or `auto`), which keeps full speed and inlining. Use `std::function` when you need to *store* callables whose types differ, such as callbacks or a task list.

```
template <typename F>
void repeat(int n, F f) { for (int i{0}; i < n; ++i) f(i); }   // fast: f can be
    inlined

repeat(3, [](int i) { std::cout << i << ' '; });                // 0 1 2
```

Chapter 18

Move Semantics and Smart Pointers

18.1 The problem: needless copies

Consider a function that builds and returns a large vector, or code that puts a long string into a container:

```
std::vector<std::string> names;
std::string s(1'000'000, 'x');      // a 1 MB string
names.push_back(s);                 // copies 1 MB: s is still needed afterwards?
    Maybe not.
names.push_back(makeString());      // copies a temporary that is about to die anyway
```

In the second call, the temporary returned by `makeString()` is destroyed right after the copy. Copying 1 MB only to throw the original away is wasteful. It would be far cheaper to *steal* the temporary's internal buffer: copy one pointer and a size, and leave the temporary empty. That is what *moving* means.

Key idea

Copying duplicates the resource; both objects own equal, independent data.
Moving transfers the resource; the destination takes over the data, and the source is left valid but (usually) empty. Moving a `std::vector` or `std::string` costs a few pointer assignments, whatever its size.

18.2 Rvalue references

To move safely, the compiler must know when an object is a temporary that nobody can use again. That is exactly an *rvalue* (Section 4.7). C++11 added a reference type that binds only to rvalues: the **rvalue reference**, written `T&&`.

```
void take(const std::string& s);   // #1: binds to anything; must copy if it keeps it
void take(std::string&& s);        // #2: binds only to temporaries; may steal from s

std::string name{"Alice"};
take(name);               // name is an lvalue  -> #1
take(name + "!");         // a temporary        -> #2
take(std::string{"x"});   // a temporary        -> #2
```

Overloading on `const T&` versus `T&&` lets a class copy from objects that are still in use and move from objects that are about to die.

18.3 Move constructor and move assignment

Return to the `Buffer` class from Chapter 8, which owns a heap array:

```
class Buffer {
public:
```

```
    explicit Buffer(std::size_t n) : size_{n}, data_{new int[n]{}} {}
    ~Buffer() { delete[] data_; }

    Buffer(const Buffer& other)                          // copy: allocate and duplicate
        : size_{other.size_}, data_{new int[other.size_]} {
        std::copy(other.data_, other.data_ + size_, data_);
    }

    Buffer(Buffer&& other) noexcept                      // MOVE constructor: steal
        : size_{other.size_}, data_{other.data_} {
        other.size_ = 0;
        other.data_ = nullptr;      // so other's destructor deletes nothing
    }

    Buffer& operator=(const Buffer& other) {             // copy assignment
        if (this != &other) {
            Buffer tmp{other};                           // copy first (may throw)
            swap(tmp);                                   // then commit
        }
        return *this;
    }

    Buffer& operator=(Buffer&& other) noexcept {         // MOVE assignment
        if (this != &other) {
            delete[] data_;                              // release what we own
            data_ = other.data_;                         // take other's resource
            size_ = other.size_;
            other.data_ = nullptr;
            other.size_ = 0;
        }
        return *this;
    }

    void swap(Buffer& other) noexcept {
        std::swap(size_, other.size_);
        std::swap(data_, other.data_);
    }
private:
    std::size_t size_;
    int* data_;
};
```

The essential step in moving is resetting the source (`other.data_ = nullptr`). Without it, both objects would own the same array and it would be deleted twice.

18.3.1 The moved-from state

After a move, the source object must still be *valid*: its destructor will run, and you may assign a new value to it. But its value is unspecified. For standard library types it is typically empty, but code must not rely on that.

Use after move

After `auto b = std::move(a);`, do not read `a`'s value. You may destroy it or assign it a new value; nothing else is guaranteed. Static analysers and `clang-tidy` can warn about use-after-move.

18.4 `std::move` does not move anything

`std::move(x)` is just a cast: it turns the lvalue `x` into an rvalue, saying "I am finished with `x`; you may steal from it". The actual moving happens in the move constructor or move assignment that this selects.

```
std::string a{"a long string ..."};
std::string b{std::move(a)};     // selects the move constructor: b takes a's buffer
// a is now valid but unspecified (in practice empty)

std::vector<std::string> v;
std::string line;
while (std::getline(std::cin, line))
    v.push_back(std::move(line));   // move each line into the vector, no copies
                                    // (getline assigns line a new value next time)
```

18.4.1 When moves happen automatically

- Temporaries are rvalues, so `v.push_back(makeString())` moves without `std::move`.
- Returning a local variable: `return result;` uses copy elision, or moves if elision is not possible. Do *not* write `return std::move(result);`: it disables elision and can make things slower.

18.4.2 Sink parameters: take by value, then move

When a function stores its argument, take it by value and move it into place. Callers passing temporaries pay only moves; callers passing lvalues pay exactly one copy.

```
class Person {
public:
    explicit Person(std::string name) : name_{std::move(name)} {}
private:
    std::string name_;
};
```

18.5 The Rule of Five, and why `noexcept` matters

A class that manages a resource by hand now needs five special members: destructor, copy constructor, copy assignment, move constructor and move assignment. Better still, follow the Rule of Zero and let members such as `std::vector` and `std::unique_ptr` handle all five.

Mark move operations `noexcept`. When a `std::vector` grows, it allocates a bigger buffer and transfers the old elements. If moving an element could throw halfway through, the vector could not restore its original state, so it *copies* instead unless the move constructor is `noexcept`. Forgetting `noexcept` can silently turn cheap moves into expensive copies.

18.6 Forwarding references and `std::forward`

In a template, `T&&` with a deduced `T` is special: it is a *forwarding reference* that binds to both lvalues and rvalues. Combined with `std::forward`, it passes an argument on while preserving

whether it was an lvalue or an rvalue. This is how `emplace_back` and `std::make_unique` pass constructor arguments through unchanged.

```
template <typename T, typename... Args>
std::unique_ptr<T> makeIt(Args&&... args) {
    return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
}
```

You will rarely write this yourself, but you will meet it constantly when reading library code.

18.7 Smart pointers

A smart pointer is an object that owns a heap object and deletes it automatically: RAII for single objects. They live in `<memory>`.

18.7.1 `std::unique_ptr`: sole ownership

```
#include <memory>

auto p = std::make_unique<Widget>(arg1, arg2);   // creates a Widget on the heap
p->doSomething();                                // use like a raw pointer
(*p).doSomething();
// no delete needed: the Widget is destroyed when p goes out of scope

// auto q = p;                // ERROR: unique_ptr cannot be copied
auto q = std::move(p);        // ownership moves to q; p is now nullptr

auto arr = std::make_unique<int[]>(100);         // owns an array; uses delete[]
```

A `unique_ptr` is exactly as small and fast as a raw pointer. It cannot be copied, because two owners would both delete the object, but it can be moved, which transfers ownership. It is the default choice for owning a heap object, for example for polymorphic objects:

```
std::unique_ptr<Shape> makeShape(char kind) {
    if (kind == 'c') return std::make_unique<Circle>(1.0);
    return std::make_unique<Square>(2.0);
}
```

18.7.2 `std::shared_ptr`: shared ownership

Sometimes several parts of a program need an object, and none of them is clearly "the owner". A `shared_ptr` keeps a *reference count*: how many `shared_ptr`s currently own the object. Copying increases the count; destroying or reassigning one decreases it; when it reaches zero, the object is deleted.

```
auto a = std::make_shared<Config>();   // count = 1
{
    auto b = a;                        // count = 2: both own the same Config
    std::cout << a.use_count();        // 2
}                                      // b destroyed: count = 1
                                       // a destroyed later: count = 0, Config deleted
```

The count lives in a separately allocated *control block*. `std::make_shared` allocates the object and the control block together in one allocation, so prefer it to `std::shared_ptr<T>(new T)`. The count is updated atomically (safely across threads), which makes copying a `shared_ptr` noticeably more expensive than copying a raw pointer.

18.7.3 `std::weak_ptr`: breaking cycles

Reference counting has one failure: cycles. If `A` holds a `shared_ptr` to `B` and `B` holds one back to `A`, each keeps the other's count above zero forever, and neither is ever freed.

A `weak_ptr` observes an object owned by `shared_ptr`s without adding to the count. To use the object, you call `lock()`, which returns a `shared_ptr` if the object still exists, or an empty one if it has been destroyed.

```
struct Node {
    std::shared_ptr<Node> next;     // owning link forward
    std::weak_ptr<Node>   prev;     // non-owning link back: no cycle
};

if (auto p = node->prev.lock()) {   // p is a shared_ptr, or empty
    use(*p);
}
```

18.7.4 Choosing

Situation	Use
One clear owner of a heap object	`std::unique_ptr` (the default)
Ownership genuinely shared, lifetime unclear	`std::shared_ptr`
Observe a shared object without keeping it alive; break cycles	`std::weak_ptr`
Look at an object you do not own, during a call	A raw pointer `T*` or reference `T&`
A collection of objects	A container such as `std::vector<T>`, not pointers at all

Best practice

Raw pointers are fine in modern C++ as long as they never *own* anything: a raw pointer means "I am looking at this, someone else is responsible for deleting it". Ownership is always expressed with a smart pointer or a container. Pass a `unique_ptr` into a function only when you are transferring ownership; otherwise pass `T&` or `T*`.

Chapter 19

The Standard Library: Containers, Iterators, Algorithms and I/O

The standard library (often called the STL, after the Standard Template Library it grew from) is a large part of why C++ is productive. Before writing a data structure or an algorithm yourself, check whether the library already has it. It usually does, and its version is well tested and fast.

19.1 Containers

Container	What it is / when to use it	Key costs
`std::vector<T>`	Growable array in one contiguous block. **The default container.**	Index $O(1)$; push_back amortised $O(1)$
`std::array<T, N>`	Fixed-size array; size known at compile time	Like a C array, but safer
`std::string`	A vector of characters with text operations	As vector
`std::deque<T>`	Double-ended queue: fast insert at both ends	Index $O(1)$
`std::list<T>`	Doubly linked list	Insert/erase $O(1)$ given an iterator; no indexing
`std::map<K, V>`	Sorted key–value pairs (balanced tree)	Lookup $O(\log n)$
`std::unordered_map<K, V>`	Hash table key–value pairs	Lookup $O(1)$ on average
`std::set<T>`, `std::unordered_set<T>`	Unique keys, sorted / hashed	As map / unordered_map
`std::priority_queue<T>`	Always gives the largest element first (a heap)	Push/pop $O(\log n)$
`std::stack<T>`, `std::queue<T>`	Adapters with a restricted interface	$O(1)$

Best practice

Use `std::vector` unless you have measured a reason not to. Its contiguous memory suits modern CPU caches so well that it often beats theoretically better structures, such as `std::list`, even for insertions in the middle.

19.1.1 `std::vector` in depth

```
std::vector<int> v;                 // empty
std::vector<int> w(5);              // five zeros          (parentheses: a count)
std::vector<int> x{5};              // one element: 5      (braces: a list of
    elements!)
std::vector<int> y(3, 7);           // 7 7 7

v.push_back(10);                    // append a copy (or move)
v.emplace_back(20);                 // construct in place from arguments
```

```
v[0];                                   // unchecked access: fast, UB if out of range
v.at(0);                                // checked access: throws std::out_of_range
v.size();                               // number of elements
v.pop_back();                           // remove the last element
v.clear();                              // remove all elements
```

A vector has a *size* (elements in use) and a *capacity* (elements it has room for). When `push_back` exceeds the capacity, the vector allocates a larger block (typically 1.5 or 2 times bigger), moves the elements across and frees the old block. Doubling makes this cheap on average, but if you know the final size, call `v.reserve(n)` up front to avoid repeated reallocations.

Iterator and reference invalidation

Reallocation moves the elements, so any pointer, reference or iterator into the vector taken before it now dangles (see the example below). Likewise, never add to or erase from a vector while iterating over it with a range-based `for` loop.

```
std::vector<int> v{1, 2, 3};
int& first = v[0];
v.push_back(4);          // may reallocate
std::cout << first;      // undefined behaviour if it did
```

19.1.2 Maps

```
std::unordered_map<std::string, int> wordCount;
for (const auto& w : words) ++wordCount[w];     // [] inserts a zero if w is missing

if (auto it = wordCount.find("apple"); it != wordCount.end())
    std::cout << it->second << '\n';            // first = key, second = value

if (wordCount.contains("pear")) { /* C++20 */ }

for (const auto& [word, count] : wordCount)     // structured bindings
    std::cout << word << ": " << count << '\n';
```

Note that `operator[]` *inserts* a default value when the key is missing, so it cannot be used on a const map. Use `find`, `contains` or `at` to look up without inserting.

19.2 Iterators

An iterator is a generalised pointer into a container. Every container provides `begin()`, which refers to the first element, and `end()`, which refers to one past the last element. The pair forms a half-open range $[\text{begin}, \text{end})$.

```
std::vector<int> v{3, 1, 4};
for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << ' ';      // dereference like a pointer
```

Range-based `for` loops are shorthand for exactly this loop. Iterators matter because algorithms are written against them, so one algorithm works with every container.

19.3 Algorithms

The `<algorithm>` and `<numeric>` headers provide well over a hundred algorithms. A few you will use constantly:

```
#include <algorithm>
#include <numeric>

std::vector<int> v{5, 3, 8, 1, 9, 2};

std::sort(v.begin(), v.end());                                  // 1 2 3 5 8 9
std::sort(v.begin(), v.end(), std::greater<>{});                // 9 8 5 3 2 1
auto it  = std::find(v.begin(), v.end(), 8);                    // iterator to 8, or
    end()
auto big = std::count_if(v.begin(), v.end(), [](int x) { return x > 4; });
int sum  = std::accumulate(v.begin(), v.end(), 0);              // note: 0, not 0.0,
    means int sum
auto [mn, mx] = std::minmax_element(v.begin(), v.end());
std::transform(v.begin(), v.end(), v.begin(), [](int x) { return x * x; });
bool anyNeg = std::any_of(v.begin(), v.end(), [](int x) { return x < 0; });
std::reverse(v.begin(), v.end());

// Erase all elements matching a condition (C++20):
std::erase_if(v, [](int x) { return x % 2 == 0; });
```

19.3.1 Ranges (C++20)

The `<ranges>` library lets you pass a whole container instead of two iterators, and compose operations lazily:

```
#include <ranges>
std::ranges::sort(v);
auto evensSquared = v | std::views::filter([](int x) { return x % 2 == 0; })
                      | std::views::transform([](int x) { return x * x; });
for (int x : evensSquared) std::cout << x << ' ';
```

Best practice

Prefer a named algorithm to a hand-written loop. `std::any_of(...)` states its intent; a loop with a flag and a `break` has to be read carefully to see what it does.

19.4 Other useful utilities

Utility	Use
`std::pair<A, B>`, `std::tuple<...>`	Group two or more values; unpack with structured bindings
`std::optional<T>`	A value that may be absent
`std::variant<A, B, ...>`	A value that is exactly one of several types (a type-safe union)
`std::string_view`, `std::span<T>`	Non-owning views of text and of contiguous arrays
`std::format` (C++20)	Python-style formatting: `std::format("{} has {:.2f}", name, x)`
`<chrono>`	Durations, clocks and timing code
`<random>`	Proper random number generation (`std::mt19937`) instead of `rand()`
`<thread>`, `<mutex>`, `<atomic>`	Concurrency
`<filesystem>`	Paths, directories and file operations

19.5 Input and output streams

19.5.1 Console and string streams

```
#include <iostream>
#include <sstream>
#include <iomanip>

int age{};
if (!(std::cin >> age)) {                // extraction fails on bad input
    std::cin.clear();                    // reset the error state
    std::cin.ignore(10000, '\n');        // discard the rest of the line
}

std::string line;
std::getline(std::cin, line);            // whole line, including spaces

std::istringstream in{"3 4.5 hello"};    // read from a string like from cin
int i; double d; std::string w;
in >> i >> d >> w;

std::ostringstream out;                  // build a string with <<
out << std::fixed << std::setprecision(2) << 3.14159;
std::string s = out.str();               // "3.14"
```

19.5.2 Files

```
#include <fstream>

{
    std::ofstream out{"prices.csv"};
    if (!out) throw std::runtime_error("cannot open file");
    out << "date,price\n" << "2026-11-06,101.5\n";
}                                        // file closed automatically (RAII)
```

```
std::ifstream in{"prices.csv"};
std::string row;
while (std::getline(in, row)) {
    std::cout << row << '\n';
}
```

File streams close themselves in their destructors, so you rarely need to call `close()`.

19.6 Quick reference: big-O of common operations

Operation	vector	deque	map / set	unordered_map / set
Access by index	$O(1)$	$O(1)$	—	—
Find by value/key	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$ average
Insert/erase at end	$O(1)$*	$O(1)$	—	—
Insert/erase at front	$O(n)$	$O(1)$	—	—
Insert/erase in middle	$O(n)$	$O(n)$	$O(\log n)$	$O(1)$ average
Ordered iteration	as stored	as stored	sorted	no order

*amortised

Chapter A

From Design to Working Code

The CS20006 course ends with software engineering: development life cycles, UML and testing. This appendix connects those ideas to the C++ you have learned.

A.1 Development life cycle models in brief

Model	Idea
Waterfall	Requirements, design, implementation, testing, maintenance, one after another. Simple, but late feedback.
V-model	Waterfall where each design stage is paired with a matching test stage (unit tests for detailed design, system tests for requirements).
Spiral	Repeated cycles of planning, risk analysis, building and evaluation; suits large, risky projects.
Agile	Short iterations (often 1–2 weeks) delivering working software, with continuous feedback and changing requirements. Most software teams today work in some agile style.

A.2 UML class diagrams and their C++ meaning

UML notation	C++
`+` / `-` / `#` before a member	`public` / `private` / `protected`
Italic class or operation name	Abstract class / pure virtual function
Solid line, hollow triangle (generalisation)	Public inheritance: `class Circle : public Shape`
Dashed line, hollow triangle (realisation)	Implementing an interface (abstract base class)
Filled diamond (composition)	A data member owned and destroyed with the whole: `Engine engine_;` or `std::unique_ptr`
Hollow diamond (aggregation)	A part that can outlive the whole: a non-owning pointer or reference, or `shared_ptr`
Plain line (association)	One class knows about another, often through a pointer or ID
Dashed arrow (dependency)	One class uses another temporarily, e.g. as a function parameter
Underlined member	`static` member

A.3 Testing your code

A.3.1 Assertions

`assert` from `<cassert>` checks a condition you believe is always true, and aborts with the file and line if it is not. Use it for internal invariants, not for validating user input.

```
#include <cassert>
double safeSqrt(double x) {
```

```
    assert(x >= 0 && "safeSqrt needs a non-negative argument");
    return std::sqrt(x);
}
```

A.3.2 Unit tests

A unit test checks one small piece (a function or a class) in isolation. Professional C++ projects use a test framework such as GoogleTest or Catch2:

```
#include <gtest/gtest.h>

TEST(ComplexTest, AdditionWorks) {
    Complex a{1, 2}, b{3, 4};
    EXPECT_EQ(a + b, Complex(4, 6));
}

TEST(StackTest, PopOnEmptyThrows) {
    Stack<int> s;
    EXPECT_THROW(s.pop(), std::underflow_error);
}
```

- **Black-box testing** chooses test cases from the specification: typical values, boundary values (0, 1, the maximum, empty containers) and invalid inputs.
- **White-box testing** chooses cases from the code's structure, aiming to execute every branch at least once (*branch coverage*).
- Write a failing test that reproduces each bug before fixing it, so the bug can never silently return.

Chapter B

Everyday Best-Practice Checklist

1. Compile with `-Wall -Wextra -Wpedantic`, and with sanitizers while developing. Fix every warning.
2. Initialise every variable, preferably with braces.
3. Make everything `const` that can be; make compile-time constants `constexpr`.
4. Pass cheap types by value and everything else by `const&`.
5. Never write `new` or `delete` in application code. Use containers and `std::make_unique`.
6. Follow the Rule of Zero; if you must manage a resource by hand, follow the Rule of Five.
7. Mark single-argument constructors and conversion operators `explicit`.
8. Mark member functions `const` when they do not modify the object.
9. Give polymorphic base classes a virtual destructor; write `override` on every override.
10. Handle polymorphic objects through references or (smart) pointers, never by value.
11. Use `static_cast` and friends, never C-style casts.
12. Throw by value, catch by `const&`; mark moves, swaps and destructors `noexcept`.
13. Use `std::vector` by default; prefer standard algorithms to raw loops.
14. Never write `using namespace std;` in a header.
15. Never return a reference or pointer to a local variable; do not keep references into a vector across a `push_back`.
16. Keep functions short and give them names that say what they do.

Chapter C

Where to Go Next

C.1 Practice projects

1. A `Matrix` class with operator overloading, then make it a template `Matrix<T>`.
2. Your own simplified `Vector<T>`: implement the Rule of Five, `reserve`, `push_back` and growth. This teaches more about C++ than any book chapter.
3. A shape hierarchy with an abstract base, stored in `std::vector<std::unique_ptr<Shape>>`.
4. A limit order book with `std::map` for price levels.
5. Rewrite a small Python script of yours in C++ and compare speed.

C.2 References

- learncpp.com: free, thorough and up to date. The topic structure of this guide follows it; read its lessons for more depth and quizzes.
- cppreference.com: the precise reference for the language and library.
- IIT Kharagpur CS20006 Software Engineering (Spring 2024) course page and slides: `https://cse.iitkgp.ac.in/~sourangshu/coursefiles/cs20202_2024s.html`.
- Bjarne Stroustrup, *A Tour of C++* (3rd ed.) and *The C++ Programming Language* (4th ed.).
- Scott Meyers, *Effective Modern C++*.
- The C++ Core Guidelines: `https://isocpp.github.io/CppCoreGuidelines/`.

All explanations and examples in this guide were written for it; for anything here that you want to explore further, the sources above are where to look.